Exact Binary Polynomial Multiplication from Fast Pascal Transforms

Árpád Csizmadia
arpad.csizmadia.public@gmail.com

31 October 2026
Version 0.1.0-preprint. Apache-2.0.

Abstract

We give an exact algorithm for multiplication in $\mathbb{F}_2[X]$ with $O(n \log(n+2))$ bit complexity, using the arbitrary-width tape interchange theorem of OpenAI (2026) as an external algorithmic input. The construction uses an explicit tower of ternary cyclotomic fields, arbitrary prefixes of an $\mathbb{F}_4$ flag, sparse polynomial-radix extraction, and a shared Taylor layer for sibling cosets. Local scalar actions are charged by the child field degree, even though the ambient coefficient field stays fixed throughout the recursion. Elementary Karatsuba multiplication suffices. A residue pairing reconstructs the product from one affine evaluation grid and its three field conjugates; its ordinary binary transpose is explicit. We prove a uniform binary tensor bound with a fixed strict saving in tensor height from a finite ternary phase network, dirty address selectors and an explicit word over $\mathbb{F}_{16}$. This supplies the Pascal transforms required by the multiplication reduction. All costs refer to one deterministic finite-alphabet machine with a fixed finite number of tapes, including setup, movement and cleanup. The external input is a routing theorem, rather than an integer-multiplication oracle or a hypothesis on primes. Bounded C++ witnesses check evaluation/product/decoder algebra and ordinary adjoints on finite instances; they do not implement the asymptotically fast tensor machine. The same asymptotic bound holds for the ordinary binary transpose of multiplication by either fixed operand.

AI assistance and formal verification. This work was developed with substantial generative AI assistance in mathematical exploration, algorithm and proof development, writing, coding, and literature review. OpenAI Codex [13] assisted release preparation and source inspection. Historical model versions were not consistently recorded; no complete model-level attribution is claimed. Full Lean formalization is unfinished and planned for a later update. Partial Lean work in the broader project is not included here and does not verify the complete theorem. Finite computational checks cover only their stated scopes. This manuscript is a working preprint open for mathematical review.

Contents

1 Introduction 2

2 Machine model and reduction statement 4

3 Explicit fields and arbitrary prefix flags 5

4 Sparse canonical radix conversion 6

5 Sibling fusion, local scalar costs and evaluation **7**
5.1 Evaluate the outer prefix before translating the inner rows . . . 7
5.2 Native child fields, even for large ambient payloads . . . 7
5.3 The full recurrence and all transposes . . . 9

6 A residue decoder and the raw product **10**
6.1 The parent is a quotient ring, not a field . . . 10
6.2 Convert the pairing to the actual binary transpose . . . 11
6.3 One padding and the complete bit bound . . . 12

7 A uniform fixed-tape bound for the binary tensor transform **12**
7.1 Opaque records and packed address shears . . . 13
7.2 Dirty active-rank selection . . . 15
7.3 Prefix-local calls and exact exceptional correction . . . 16
7.4 Ternary phase frame and fixed role recursion . . . 17
7.5 Internal payload and four-block layout . . . 19
7.6 A fixed positive word and completion of the tensor theorem . . . 20

8 Exact computational witnesses and reproducibility **21**

A Finite role and phase certificate **22**
A.1 The scalar eight-gate schedule . . . 22
A.2 Array frames and a fixed-tape compiler . . . 23
A.3 Factor geometry and gate labels . . . 24
A.4 Phase directions at a growing node . . . 26

B An explicit positive word for the binary tensor core **27**

1 Introduction

Let $F, G \in \mathbb{F}_2[X]$ have degree less than n. Their product has at most $2n-1$ binary coefficients, so the input and output have total size $\Theta(n)$. We study the bit complexity of computing this product on a deterministic multitape Turing machine with a fixed finite alphabet and a fixed finite number of one-dimensional tapes. Every tape scan, data rearrangement, setup operation, and output step is included in the time bound. The target is an exact algorithm, uniform over all input lengths.

For fixed prime fields, our unconditional comparison is Harvey and van der Hoeven's cyclotomic-coefficient-ring algorithm. For each fixed prime p, it multiplies degree-$< n$ polynomials in $\mathbb{F}_p[X]$ using $O(n \log n\, 4^{\log^* n})$ bit operations on a deterministic multitape Turing machine [3]. In particular, this applies to $p = 2$. The bound obtained here from the cited tape-interchange theorem removes that unbounded iterated-logarithmic factor for this fixed field. For fixed p, an earlier result of Harvey, van der Hoeven, and Lecerf gives $O_p(n \log n\, 8^{\log^* n})$ bit operations [5]; for fixed p, the 2019 theorem is the sharper comparison.

There is also a conditional finite-field theorem. Harvey and van der Hoeven prove, uniformly over finite fields of size q, a bit bound $M_q(n) = O(n \log q \log(n \log q))$ under the hypothesis that a Linnik constant exists with $L < 1 + 2^{-1162}$ [4, Theorem 1.2]. This result does not supply the unconditional fixed-$\mathbb{F}_2$ bound considered here. Separately, Harvey and van der Hoeven prove unconditional $O(n \log n)$ bit complexity for integer multiplication in the multitape Turing model [6]; integer multiplication is not itself the finite-field polynomial problem. The September 2026 OpenAI preprint claims a faster exact integer multiplier in a fixed finite-tape model, and proves, among other tools, an equal-chunk interchange bound in its Lemma 12 [7,

Lemma 12, Section 4]. We use that lemma only as a machine-level interchange dependency. The preprint does not state the finite-$\mathbb{F}_2$ polynomial multiplication theorem of this paper.

Harvey's October 2026 discussion also singles out fixed-field polynomial multiplication as a separate problem from the new integer claims [12, Section 4]. We use that discussion to locate the problem, without suggesting endorsement or verification of this manuscript.

The earlier work established the arithmetic, evaluation and bit-cost framework for the present question. Reaching $n \log n$ without the number-theoretic assumption in the fixed-field case is the baseline advance claimed here. We pursue it by adapting the recent finite-network tape method to explicit binary evaluation and reconstruction. The prior uniform-field and lookup-table results retain their own broader scopes or different access models; the comparison here concerns ordinary binary multiplication on fixed sequential tapes.

The proof strategy has two logically distinct parts. First, we give an explicit reduction from polynomial multiplication to a uniform fast binary tensor action. The algebraic part builds a tower of ternary cyclotomic fields and uses arbitrary prefixes of an $\mathbb{F}_4$-linear flag. Sparse polynomial-radix extraction is expressed through Pascal transforms; sibling-coset evaluation shares its Taylor stages. Local scalar operations are charged in the child field, while coefficient words retain their full ambient width. The product is reconstructed from one affine evaluation grid and its field conjugates using an explicit residue pairing. The proof also constructs the ordinary binary transpose of the fixed-operand map directly, with its own cost accounting.

Second, we derive the needed tensor action from a finite construction: an exact ternary phase network, a fixed word over $\mathbb{F}_{16}$, and a router with opaque payloads and dirty selectors. The sole external machine theorem is the equal-chunk interchange result just described. The scalar bank exchange and array-frame compiler used in the local adaptation are proved in Appendix A, with their source attribution retained. The reduction and the tensor theorem are stated separately: the reduction needs any uniform strict saving in tensor height, whereas the tensor proof supplies such a saving and accounts for finite words, routing, arbitrary payload, and fixed tapes.

Evaluation on additive subspaces has a substantial history, including Cantor's constructions [8] and Gao–Mateer's Taylor-expansion-based additive FFTs [9]. Liu's generalization gives an $O(n \log n (\log\log n)^2)$ binary polynomial bound [10]; this is relevant related work, although the 2019 cyclotomic bound is asymptotically sharper. Evaluation and interpolation on structured sets, and their interpretation as basis conversions, are classical topics; see also Bostan and Schost [1]. The present issue is the complete bit cost of the particular prefix conversions, their scalar actions and their tape layouts.

Sub-$N \log N$ bit bounds for binary tensor transforms also have prior art. Alman [2, Section 1.1, Section 1.2 and footnote 4] gives $O(N \log N / \log\log N)$ bit operations for fixed-matrix Kronecker powers in a RAM model with lookup tables; the paper explicitly includes the kernel $H_2 = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)$. This supplies an important comparison, but does not state the fixed-tape, arbitrary-payload contract used here. Our target is a fixed exponent $\beta < 1$ in $O(N(\log N)^\beta)$ with movement and cleanup included. The fixed-power saving matters to the evaluator recurrence: a toll $k/\log k$ in its balanced version sums to $\Theta(k \log\log k)$, whereas a toll k^β sums to $O(k)$.[1] We make no claim of the first sub-$N \log N$ binary tensor algorithm.

The bounded C++ witnesses accompanying the manuscript check finite instances of the evaluation, product, decoder, and adjoint identities. They do not implement the asymptotically fast tensor machine; the running-time bound rests on the uniform proof. No practical-speed or priority claim is made. The following sections specify the machine model, define the field and flag coordinates, prove the radix and evaluation identities, construct the decoder and adjoints, and then prove the tensor theorem and describe the finite witnesses.

[1] For $k = 2^a$, the nonbase levels of the balanced tree contribute $k \sum_{j=1}^{a} 1/j$ to a $k/\log_2 k$ toll. This observation concerns that recurrence, and does not rule out other uses or refinements of lookup-table algorithms.

2 Machine model and reduction statement

An input polynomial of degree less than n is its n coefficients in ascending degree order, including leading zero coefficients. The output has $2n-1$ coefficients. Time counts steps of one deterministic Turing machine with a fixed finite alphabet and a fixed finite number of one-dimensional tapes. The machine, its tapes and its finite tables do not depend on n. All bit movement, setup and cleanup are included. The number of fixed tapes and the constants may be extremely large. This paper makes no claim for a specified two-tape machine or for practical execution time.

Internal array interfaces receive a self-delimiting binary description of their shape on descriptor tapes. In particular the tensor interface receives t, D, and a batched evaluator receives k, D, B_0; parameter processing and counter initialization are included in the bounds. These are shape parameters computed by the calling algorithm, with no input-dependent advice. At the top, an initial scan determines the raw input length in $O(n)$ steps. A shape word of length $O(\log(D+1))$ does not have unit cost; its handling is charged to the nonempty D-bit records as specified in the subroutines.

For a binary linear map L, L^{T} always denotes the transpose in the displayed coefficient coordinates. Field-trace adjoints will be converted to this transpose by explicit Gram maps. No generic circuit-transposition or inverse-as-adjoint simulation is assumed.

Contract 2.1 (Equal-chunk interchange)**.** *For a fixed $0 < \tau < 1$, the permutation*

$$[P] \times [2^u] \times [G] \times [2^u] \times [B] \longrightarrow [P] \times [2^u] \times [G] \times [2^u] \times [B], \quad (p, x, g, y, z) \longmapsto (p, y, g, x, z)$$

has time $O(Vu^\tau)$, where $V = PGB2^{2u}$. The constant is uniform in all positive dimensions, including $B = 1$, and includes processing the shape, temporary padding, unpadding and cleanup.

This is the statement imported from Lemma 12 of [7]. It is an external theorem dependency, not a conjecture about primes or about polynomial multiplication. Section 7 explains its separate use in the tensor construction.

Contract 2.2 (Uniform binary tensor transform)**.** *For some fixed $\beta < 1$, put*

$$H = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}.$$

For every $t \geq 1$ and binary payload width $D \geq 1$, one fixed machine computes $H^{\otimes t} \otimes I_D$ in $O(2^t D t^\beta)$ time, including all auxiliary representations, padding and cleanup. The constant is independent of D.

Since $H = H^{\mathsf{T}}$ and $H^3 = I$, this same algorithm computes the binary transpose, and two applications compute the inverse. If $S = \left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$ and $J = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$, then $S = HJ$. Lucas' binomial congruence gives the Pascal matrix

$$(\mathcal{P}_{2^t})_{s,j} = \binom{j}{s} \bmod 2 = (S^{\otimes t})_{s,j}.$$

Thus one call to Contract 2.2 and reversal of the record order compute $\mathcal{P}_{2^t}$. Its transpose places that reversal on the other side. The payload can be a list of coefficient words: it need not be one element of an enormous field.

Lemma 2.3 (Rectangular reorientation)**.** *Contract 2.1 permits the address-field rotation $[2^a] \times [2^b] \to [2^b] \times [2^a]$ in $O(V(a+b)^\gamma)$ time, where $\gamma = 1/(2-\tau) < 1$. The bound includes an arbitrary opaque suffix payload.*

Proof. Use the Euclidean block-rotation schedule. At widths $a \geq b$, write $a = qb+r$, exchange the right field past the q complete b-bit chunks, then rotate the remaining pair of widths b, r. Stop when the smaller positive width is less than z; move its individual bits across the other field, preserving their order. Single-bit moves cost $O(V)$ by a sequential split and merge. The Euclidean remainders satisfy

$$\sum_i q_i r_i^\tau \leq z^{\tau-1} \sum_i q_i r_i \leq (a+b) z^{\tau-1}.$$

The last sum telescopes, since $q_i r_i = r_{i-1} - r_{i+1}$. The final individual-bit moves cost $O(Vz)$. Choose z within a constant factor of $(a+b)^\gamma$ using a power of two and integer arithmetic on $\lceil \log_2(a+b) \rceil$. Both terms have the asserted bound. Reverse the actual schedule for the transpose permutation. □

Theorem 2.4 (Exact reduction)**.** *Contracts 2.1 and 2.2 imply an exact $O(n \log n)$ algorithm for multiplying two degree-$< n$ polynomials over $\mathbb{F}_2$ in the stated model. For either fixed operand, the ordinary binary transpose of the multiplication map has the same bound.*

Sections 3–6 prove this implication. The removal of Contract 2.2 as a separate premise is isolated in Section 7 and Appendix B. Theorem 2.4 does not assume an arithmetic-operation model with unit-cost extension-field scalars.

3 Explicit fields and arbitrary prefix flags

For $K \geq 2$, let A be the largest power of three strictly smaller than K, so $A < K \leq 3A$, and put

$$\begin{aligned} b &= 4^K, \qquad B = \mathbb{F}_2[Z]/\Phi_{9A}(Z), \quad w = 6A, \\ E &= \mathbb{F}_2[Y]/\Phi_{27A}(Y), \quad D = 18A, \qquad Z = Y^3. \end{aligned} \tag{1}$$

Here and below Φ is a cyclotomic polynomial, reduced modulo two. The fields are given in their native monomial bases.

Lemma 3.1. *If m is a power of three, then $\Phi_{3m}(T) = T^{2m} + T^m + 1$ is irreducible over $\mathbb{F}_2$. The chart in (1) is an explicit degree-three extension E/B. With $\eta = Y^{9A}$, its relative Frobenius sends Y to ηY, where $\eta^2 + \eta + 1 = 0$.*

Proof. The order of 2 modulo 3^{j+1} is $2 \cdot 3^j$. For example, $v_3(4^{3^j} - 1) = j+1$, proved inductively by expanding $(1 + 3^{j+1}c)^3$, proves the order assertion. Thus the order equals $\varphi(3m) = 2m$, which proves the cyclotomic irreducibility. Substitution gives $\Phi_{27A}(Y) = \Phi_{9A}(Y^3)$. The identity $4^{3A} \equiv 1 + 9A \pmod{27A}$ gives the displayed relative Frobenius. In particular $1, \eta, \eta^2$ are the specified four-element subfield, with $\eta^2 = 1 + \eta$. □

Write $Q = 4^a$ for a power of three a, and let u be the native root of order $9a$ in $\mathbb{F}_{Q^3}$. Then

$$\mathbb{F}_{Q^3} = \mathbb{F}_Q \oplus u\mathbb{F}_Q \oplus u^2\mathbb{F}_Q, \qquad u^Q = \eta u.$$

Starting from the basis (1) over $\mathbb{F}_4$, recursively order each cubic extension basis in three consecutive blocks:

$$(\psi_i)_i, \qquad (u\psi_i)_i, \qquad (u^2\psi_i)_i.$$

Explicitly, let $\mathcal{E}_1 = (0)$ and define ordered exponent lists by

$$\mathcal{E}_{3a} = (3\mathcal{E}_a,\ 3\mathcal{E}_a + 1,\ 3\mathcal{E}_a + 2).$$

The lower native root is u^3, so the first block u^{3e} is exactly the previously defined lower-field basis, followed by its u and u^2 multiples. These are ternary digit-reversed native powers. In

the fixed ambient chart the node root is explicitly $u = Y^{3A/a}$, so $u^{3a} = \eta$ at every node. The recursive order is therefore consistent under the actual native field inclusions. Let W_k be the initial k-dimensional $\mathbb{F}_4$ subspace in the smallest such field containing it. In particular $W_K \subseteq B$ and $1 \in W_K$.

For a node $k > 1$, take the largest power of three $a < k$ and define

$$(m, r, l) = \begin{cases} (a, k-a, 1), & k \le 2a, \\ (2a, k-2a, 2), & 2a < k \le 3a. \end{cases} \qquad M = 4^m, \quad \ell = 4^r. \tag{2}$$

Thus $1 \le r \le a$ and $m \ge k/2$. No prefix is padded to the next full field dimension.

Lemma 3.2 (Sparse low-prefix annihilators)**.** *For $Q = 4^a$, the annihilator of W_m and the upper image scale are*

$$P_1(X) = X^Q + X, \qquad c_1 = P_1(u) = \eta^2 u,$$
$$P_2(X) = X^{Q^2} + \eta^2 X^Q + \eta X, \quad c_2 = P_2(u^2) = \eta u^2.$$

For the split (2), $W_k = W_m \oplus u^l W_r$ and $P_l(x + u^l v) = c_l v$ for $x \in W_m$, $v \in W_r$.

Proof. $W_a = \mathbb{F}_Q$ and $W_{2a} = \mathbb{F}_Q \oplus u\mathbb{F}_Q$ by the recursive basis order. The displayed polynomials vanish on these spaces: use $u^Q = \eta u$ and $u^{Q^2} = \eta^2 u$. They are monic and have the correct degree and nonzero derivative, so these are all their roots. Every $v \in W_r$ belongs to $\mathbb{F}_Q$, and Q-powering fixes v. Consequently $P_l(u^l v) = P_l(u^l)v = c_l v$. Linearity proves the rest. □

The coefficients transformed recursively are arbitrary elements of the same ambient E, of binary width D, throughout. The flag describes evaluation points, not a promise that data lie in a smaller coefficient field. The bounded case $k = 1$ is the four-point $\mathbb{F}_4$ Vandermonde map, with fixed scalar matrices on native words. Its constants are $0, 1, \eta, \eta^2$; native root shifts and XORs implement each in $O(D)$ time. This base is uniform in arbitrarily large ambient width D.

4 Sparse canonical radix conversion

Let $P = X^M + h$, where h has at most two monomials with coefficients in $\mathbb{F}_4$, and $d = \deg h$ satisfies $d\ell \le M$. Write $f = \sum_{t<\ell} f_t X^{Mt}$, with $\deg f_t < M$.

Lemma 4.1 (Canonical digits and synthesis)**.** *Define*

$$F_s = \sum_{t \ge s} \binom{t}{s} f_t h^{t-s} = L_s + X^M B_s, \qquad \deg L_s < M.$$

With $B_{-1} = B_{\ell-1} = 0$, the canonical P-radix digits are

$$c_s = L_s + hB_s + B_{s-1}, \qquad f = \sum_{s<\ell} c_s P^s, \qquad \deg c_s < M. \tag{3}$$

Conversely, apply the same row Taylor map to (c_s), obtaining $G_t = L'_t + X^M H_t$. The ordinary monomial blocks are

$$f_t = L'_t + H_{t-1}. \tag{4}$$

Proof. Substitute $X^M = P + h$ and expand by the binomial theorem. This gives $f = \sum_s F_s P^s$. Replace $X^M B_s$ by $(P + h)B_s$ and collect equal powers of P to obtain (3). The degree bound

$$\deg F_s < M + d(\ell - 1 - s), \qquad \deg B_s < d(\ell - 1 - s)$$

implies $\deg(hB_s) < d(\ell - s) \le M$. For synthesis expand $P^s = (X^M + h)^s$, giving $\sum_t G_t X^{Mt}$, and collect adjacent high and low blocks. This proves (4); there is no hH_t term in it. □

The condition holds for both polynomials in Lemma 3.2: for P_1, $d = 1$ and $\ell \le Q = M$; for P_2, $d = Q$ and $\ell \le Q$, whereas $M = Q^2$.

For a monomial $h = \alpha X^e$, its row Taylor operation factors as

$$(F_s)_s = \mathrm{diag}(\alpha^{-s} X^{-es})\, \mathcal{P}_\ell\, \mathrm{diag}(\alpha^t X^{et})\, (f_t)_t. \tag{5}$$

The negative shifts mean deletion of known zero prefixes, not Laurent coefficients. Indeed every contributing index t obeys $t \ge s$. If $h = h_1 + h_2$, compose the commuting Taylor translations for the two monomials. Each intermediate row has $O(M)$ coefficients: even the crude bound using both shifts is below $3M$. A common $4M$-slot rectangle therefore suffices, with a constant volume factor that is removed before any recursive evaluator call.

Store complete coefficient words contiguously. Row shifts, fixed $\mathbb{F}_4$ scalings and the carry operation cost $O(M\ell D)$. Lemma 2.3 reorients the row/coefficient address fields at cost $O(M\ell D k^\gamma)$, where their total address width is $O(k)$, not merely $O(r)$. On that rectangle the Pascal map has payload equal to the entire coefficient row and is covered by Contract 2.2. Alternatively this is the direct uniform-payload interface to that contract. Both layouts include their physical rearrangements.

All transposes are explicit coordinate operations. If a row map is written $R\mathcal{P}_\ell G$, its transpose is $G^\mathsf{T}\mathcal{P}_\ell^\mathsf{T} R^\mathsf{T}$: R^T inserts zeros at the deleted prefix and G^T selects the original input interval. The carry map (3) sends a covector z_s to z_s on L_s and $M_h^\mathsf{T} z_s + z_{s+1}$ on B_s, with the appropriate degree projections. Synthesis sends z_{s+1} to H_s. The transpose of multiplication by a fixed monomial h is just a reverse shift and fixed-field scalar transpose. These statements hold for arbitrary covectors: the primal zero-prefix fact is never imposed on a cotangent.

5 Sibling fusion, local scalar costs and evaluation

5.1 Evaluate the outer prefix before translating the inner rows

After Lemma 4.1, scale digit t by c_l^t and, for each inner monomial coefficient, evaluate the outer polynomial on W_r. This produces the degree-$< M$ polynomials

$$g_v(X) = \sum_{t<\ell} c_t(X) c_l^t v^t, \qquad v \in W_r. \tag{6}$$

For $x \in W_m$, Lemma 3.2 gives $f(x + u^l v) = g_v(x + u^l v)$. Hence translate g_v by $\delta_v = u^l v$ and evaluate the result on W_m. For $v = 0$ the translation is the identity. For every $v \ne 0$,

$$\mathrm{Taylor}_{\delta_v} = \mathrm{diag}(\delta_v^{-s})\, \mathcal{P}_M\, \mathrm{diag}(\delta_v^t). \tag{7}$$

This is the coefficient identity $[X^s] g(X + \delta) = \sum_{t \ge s} \binom{t}{s} g_t \delta^{t-s}$.

Thus there are two scalar actions per coefficient in the translation, plus the separate outer scale. There is no scalar action at every Pascal butterfly level. Evaluating (6) first is important: δ_v is now a public field element, so no symbolic two-variable coefficient box is expanded when $r \ll m$. The zero row bypasses (7), and its transpose bypasses it by the same identity map. In particular no inverse of zero is requested.

5.2 Native child fields, even for large ambient payloads

At the node with macro power a, all phases and scales belong to

$$B_a = \mathbb{F}_2[T]/\Phi_{9a}(T), \qquad d = 6a = \Theta(k).$$

This field has $\mathbb{F}_4$ dimension $3a$, regardless of whether $m = a$ or $m = 2a$. It would be incorrect to demand a subfield of dimension $3m$ in the second case. Let $h_0 = 3A/a$; the powers of three divide, and

$$T = Y^{h_0}, \qquad \Phi_{27A}(Y) = \Phi_{9a}(Y^{h_0}), \qquad E = \bigoplus_{j<h_0} Y^j B_a, \quad D = h_0 d. \tag{8}$$

This chart is the literal coefficient permutation $i = j + h_0 t$.

Lemma 5.1 (Uniform short scalar action)**.** *Put $\alpha = \log_2 3$ and $\rho = \alpha - 1 < 1$. Multiplication by a public element of B_a on an arbitrary E word, and its ordinary binary transpose, cost $O(Dk^\rho)$, uniformly even when D/k is arbitrarily large.*

Proof. Pack the $d \times h_0$ coefficient rectangle in $O(D \log \min(d, h_0))$ time. A fixed-tape construction recursively bisects the shorter dimension, transforms its two parts and merges corresponding row fragments from two sequential streams. At every depth the total moved volume is $O(D)$ and there are $O(\log \min(d, h_0))$ depths. The inverse and the actual transpose permutation have the same cost. Record-length counters use amortized ripple updates; parameter initialization is at most the volume of the current stream, not a $\log D$ charge per bit.

Binary polynomial Karatsuba multiplication [11] costs $K(d) = O(d^\alpha)$ on fixed tapes. Pad once per local product to the next power of two. Three half-size products, sequential XORs and depth-first parking give $K(2s) \le 3K(s) + O(s)$. Parent storage is parked locally and not crossed by a descendant. The native modulus is $T^{2s} + T^s + 1$, with $d = 2s$. A raw product split into four aligned s-bit blocks reduces by

$$A_0 + T^s A_1 + T^{2s} A_2 + T^{3s} A_3 \longmapsto (A_0 + A_2 + A_3) + T^s (A_1 + A_2).$$

This needs $O(d)$ sequential moves and XORs. Apply the same scalar to the h_0 components in (8). The total cost is

$$O(D \log d + h_0 d^\alpha) = O(Dk^\rho).$$

For the transpose, let K_d be the coefficient-residue Gram matrix for $\lambda(a) = [T^{d-1}]a$ modulo that native modulus. Its entries are one exactly when $i + j = 2s - 1$ or $3s - 1$. Let J_d reverse d coordinates and let N copy the last s input coordinates into the first s outputs, zero elsewhere. Then

$$K_d = J_d(I + N), \qquad N^2 = 0, \qquad K_d^{-1} = (I + N)J_d.$$

Associativity of $\lambda(ab)$ gives

$$M_c^{\mathsf{T}} = K_d M_c K_d^{-1}. \tag{9}$$

Thus two explicit linear-cost maps and one actual scalar product compute the transpose, with the same bound. Transpose the packing permutation literally. This proof uses no inverse-Frobenius rule and no trace-adjoint substitution for an ordinary coordinate transpose. □

The positive and negative powers in (7) are generated sequentially in the d-bit field, for each outer label v. There are $O(M\ell)$ such short products. Their time $O(M\ell d^\alpha)$ is at most $O(M\ell Dk^\rho)$. One field inversion per nonzero v may use any fixed polynomial-time algorithm in k: $M \ge 4^{k/2}$ absorbs this per-label setup. Fixed small k uses direct fixed-subfield maps on each native word. Likewise the basis powers, native indices and inverses of the scales c_l are public short-field computations. No generic ambient-field product is used at a small recursive node.

The separate outer scale in (6) is charged as well. Generate the ℓ powers c_l^t with $O(\ell)$ products in B_a, then apply them to the $M\ell$ native E words using Lemma 5.1. The total is

$$O(\ell d^\alpha + M\ell Dk^\rho) = O(M\ell Dk^\rho).$$

The ordinary transpose uses the same local Gram formula. With an external batch of B_0 independent lanes the data-action cost is multiplied by B_0, while the public powers are generated once.

5.3 The full recurrence and all transposes

Choose a fixed

$$\max(\beta, \gamma, \rho) < \theta < 1. \tag{10}$$

Let $F_D(k)$ be the time to evaluate an arbitrary degree-$< 4^k$ polynomial over E on W_k, with the fixed top width D. Radix extraction, outer scaling, the shared Pascal/Taylor layer, public setup and rectangular reorientation together cost $O(4^k Dk^\theta)$. Equations (6)–(7) therefore give

$$F_D(k) \le 4^r F_D(m) + 4^m F_D(r) + C4^k Dk^\theta. \tag{11}$$

The 4^r inner evaluations are identical maps on separate coefficient lists, after the distinct translations have been performed. They can be streamed as one batched map, with the batch index an opaque payload coordinate. The same applies to the outer evaluations. Scalar actions remain componentwise on the original D-bit field words. This batching does not reinterpret a list of words as a larger field and does not reload a parent array once for every child. Copying into and out of the local child I/O interval contributes to the displayed node toll.

More precisely, strengthen the interface to $\mathrm{Eval}(k, D, B_0)$, with stream shape $[4^k][B_0][D]$ and volume $V = B_0 4^k D$. After radix extraction the shape is $[\ell][M][B_0][D]$. The outer call is $\mathrm{Eval}(r, D, B_0 M)$, consuming the whole current array. Perform the v-dependent translations, reorient the two local fields $[\ell][M]$ to $[M][\ell]$ by Lemma 2.3, and call $\mathrm{Eval}(m, D, B_0\ell)$ on that same whole array. The external lane remains an identity factor and the field degree remains D. Thus the uniform time satisfies

$$T_D(k, B_0) \le T_D(r, B_0 M) + T_D(m, B_0\ell) + CVk^\theta.$$

Normalizing by V gives the same recurrence as (11). Only descriptors are suspended on the parent stack: all current data belong to the active child, so there is no inactive sibling array to park or replay. Header initialization is charged once per call, and amortized ripple counters charge record boundaries to their scans. The local reorientation depends on k, not on the size of the external lane.

Lemma 5.2 (All-prefix recurrence)**.** *The recurrence* (11) *implies* $F_D(k) = O(4^k Dk)$ *with constants independent of the top width* D*.*

Proof. Normalize by $4^k D$ and denote the resulting cost by $T(k)$. If $k = a + r$, expand the power-three child a into three $a/3$ children using its two macro steps. The new children have heights $a/3, a/3, a/3, r$. If $k = 2a + r$, first split $2a$ into two a children and expand both, obtaining six $a/3$ children and r. All bounded cases are included directly. In each case the child heights x_i sum to k, each is at most $k/2$, and only a bounded number of tolls $O(k^\theta)$ has been introduced. Since

$$\sum_i x_i^\theta \ge (k/2)^{\theta-1} \sum_i x_i = 2^{1-\theta} k^\theta,$$

induction with the upper potential $C_0 k - Lk^\theta$ succeeds after choosing L to dominate the toll divided by $2^{1-\theta} - 1$, and C_0 to cover the fixed bases. □

The ordinary transpose reverses inner evaluations, the two actual scalar transposes in (7), the Pascal transpose, outer evaluations, scales, and radix extraction. Lemma 5.1 and the explicit shift/carry transposes give the same recurrence on unrestricted covectors. The inverse also has the same bound: invert the children and scales, use the same characteristic-two translation, then the separately proved synthesis (4).

Evaluation on the affine grid $Y + W_K$ is one initial translation by Y followed by this evaluator. Its diagonal factors are native Y^i and Y^{-s} actions. Reduce their exponents modulo $27A$; cyclic shifts in a $27A$-slot representation and an aligned fold cost $O(D)$ per word. Its

Pascal call is covered by Contract 2.2. It follows that affine evaluation and its ordinary transpose both cost $O(bDK)$. This global affine shift is performed once, not at every recursive node.

6 A residue decoder and the raw product

6.1 The parent is a quotient ring, not a field

Take the explicit fields (1) and set

$$\mathcal{R} = B[X]/(X^b - Z), \qquad N = wb = 6Ab.$$

The coordinate $X^i Z^j$ is the raw binary monomial X^{i+bj}. More formally

$$\mathcal{R} \simeq \mathbb{F}_2[X]/\Phi_{9A}(X^b),$$

and the modulus is monic of degree N. This ring is inseparable; we will use no field trace or inversion in it.

Put $W = W_K$ and let $L = L_W$ be its monic annihilator. Since W is an $\mathbb{F}_4$-space in B,

$$L(X) = X^b + C(X), \qquad \deg C \le b/4, \qquad \ell_0 = L' \neq 0.$$

Set $u_0 = L(Y) \neq 0$. Relative Frobenius fixes L and sends u_0 to ηu_0. Consequently

$$F(X) = L(X)^3 + u_0^3 \in B[X] \tag{12}$$

is monic, squarefree, of degree $3b$, and its roots are the three distinct cosets $\eta^t Y + W$, $t = 0, 1, 2$. The three grids are field conjugates; they are not three unrelated evaluation spaces.

Lemma 6.1 (Coefficient kernel with its degree cutoff)**.** *For* $\deg g < 3b - 1$*, let* $\lambda(g) = [X^{b-1}](g \bmod (X^b - Z))$*. Define polynomial quotient parts by* $[\ ]_+$ *and put*

$$\begin{aligned} Q(X) &= [F/X^b]_+ + Z[F/X^{2b}]_+ \\ &= X^{2b} + X^b C + C^2 + ZX^b + ZC. \end{aligned} \tag{13}$$

Then

$$\lambda(g) = g_{b-1} + Z g_{2b-1} = \sum_{F(\xi)=0} \frac{g(\xi)Q(\xi)}{F'(\xi)}. \tag{14}$$

Proof. For any root ξ and $1 \le j \le 3b$,

$$[X^{j-1}]\frac{F(X)}{X - \xi} = [F/X^j]_+(\xi).$$

To see this, write the quotient as a sum of the finite geometric quotients $(X^t - \xi^t)/(X - \xi)$, using $F(\xi) = 0$. Extract coefficient $j - 1$ in the Lagrange formula for g, first with $j = b$ and then $j = 2b$. The stated cutoff also excludes the otherwise possible residue contribution $Z^2 g_{3b-1}$. Expansion of (12) gives (13); the discarded terms have degree less than b because $\deg C \le b/4$. □

At the representative $\xi = Y + v$, $v \in W$, write $s = \xi^b$. Since $C(\xi) = u_0 + s$, the weight is

$$\omega_v = \frac{Q(\xi)}{F'(\xi)} = \frac{s^2 + u_0 s + u_0^2 + Z u_0}{\ell_0 u_0^2}. \tag{15}$$

Only the nonzero constant denominator is inverted.

6.2 Convert the pairing to the actual binary transpose

Let $A_B : B^b \to E^b$ evaluate on $Y + W$, including the stride-three native embedding $Z^j \mapsto Y^{3j}$. Let G_B, G_E be the absolute field trace Grams in the native coefficient bases, acting separately on every coefficient word. Let J_b reverse the coefficient records without reversing bits inside a word, and put

$$K_{\mathcal{R}} = J_b \otimes G_B.$$

This is the Gram of $(f, t) \mapsto \mathrm{Tr}_{B/\mathbb{F}_2}\, \lambda(ft)$. It is nonsingular: for $\deg f, \deg t < b$ the product has degree at most $2b-2$, so its residue coefficient $b-1$ is the ordinary anti-diagonal pairing.

Proposition 6.2 (Decoder and ordinary adjoint)**.** *Define a binary linear map on arbitrary E-valued inputs by*

$$D_{\mathcal{R}} = K_{\mathcal{R}}^{-1} A_B^{\mathsf{T}} G_E M_\omega. \tag{16}$$

For $f, g \in B[X]$ of degree less than b,

$$D_{\mathcal{R}}(A_B(f) \cdot A_B(g)) = fg \bmod (X^b - Z).$$

Its ordinary binary transpose is

$$D_{\mathcal{R}}^{\mathsf{T}} = M_\omega^{\mathsf{T}} G_E A_B K_{\mathcal{R}}^{-1}. \tag{17}$$

Proof. Apply Lemma 6.1 to fgt for arbitrary $t \in B[X]$ of degree less than b. Its degree is at most $3b-3$, so the cutoff is satisfied. Group the three conjugate summands for each v into the relative trace E/B, then apply the absolute trace of B. Trace transitivity gives precisely

$$t^{\mathsf{T}} K_{\mathcal{R}}(fg \bmod (X^b - Z)) = (A_B t)^{\mathsf{T}} G_E M_\omega (A_B f \cdot A_B g).$$

Nonsingularity of $K_{\mathcal{R}}$ proves (16). This uses the field traces of B, E, not a trace of $\mathcal{R}$. Both field Grams and $K_{\mathcal{R}}$ are symmetric, so literal matrix transposition proves (17). No restriction is imposed on an input covector to that formula. □

The field Grams have linear-time inverses without generic matrix inversion. For $g_0(T) = T^{2s} + T^s + 1$ with s a power of three, let K_{2s} be the coefficient Gram from Lemma 5.1. The separable-modulus trace identity gives

$$G = K_{2s} M_{T^{s-1}}, \qquad G^{-1} = M_{T^{1-s}} K_{2s}^{-1}. \tag{18}$$

Indeed root-by-root Lagrange interpolation yields $\mathrm{Tr}(a) = [T^{2s-1}](g_0' a \bmod g_0)$, and $g_0' = T^{s-1}$. The K maps are reversals and aligned block XORs; native root powers are cyclic shifts and folds. Thus both maps in (18) cost $O(s)$. In the bounded witnesses the equivalent form is $G = S^-$, where S swaps the two s-bit blocks and the bar substitutes T^{-1}; its inverse is bar followed by S.

There are b pointwise field products and a constant number of weight/scalar products per point in the forward and transposed maps. The same elementary Karatsuba implementation [11] costs

$$O(bD^\alpha) = O(NK^\rho), \qquad \rho = \log_2 3 - 1 < 1. \tag{19}$$

Equation (9), now in the ambient field, prices their ordinary transposes as actual field products too.

The weights in (15) are $\mathbb{F}_2$-affine functions of the $2K$ bits indexing v: both $v \mapsto v^b$ and squaring are additive. Compute the constant weight and $2K$ differences in polynomial time in K, then enumerate the table with an odometer. Binary increments flip fewer than two bits on average, giving $O(bD)$ work. The annihilator has only $K+1$ linearized coefficients; extending the flag one direction at a time computes it by $L_{j+1} = L_j^4 + L_j(\psi_j)^3 L_j$ in polynomial time in K. Its derivative, the public denominator inverse and field root indices have the same scope. There are no b generic inversions.

6.3 One padding and the complete bit bound

Choose the first $K \geq 2$ with $N_K = 6A(K)4^K \geq 2n-1$; bounded smaller inputs are handled directly. Consecutive sizes grow by at most twelve, so $N_K < 24n$ outside those fixed base cases. The raw product has degree at most $2n-2 < N_K$, hence reduction in $\mathcal{R}$ does not change it. Also $K = \Theta(\log n)$, $D = \Theta(K)$ and $bD = 3N_K$.

Let I_n zero-extend and pack the raw input into the native B^b chart, and let C_n unpack the parent result and select its first $2n-1$ bits. To pack, transpose the raw $[w][b]$ bit rectangle to $[b][w]$. Pad the short range w to the next power of two, increasing volume by less than two; its address width is $O(\log K)$, while the other width is $2K$. Lemma 2.3 gives time $O(N_K K^\gamma)$, with $\gamma < 1$. Remove the inserted coordinates and apply the stride-three embedding by a linear scan. Unpacking reverses these explicit operations. Their ordinary transposes reverse the permutation and exchange zero insertion with selection, with the same bound. The padding is in a temporary address layout and does not change either field degree. The complete algorithm and its fixed-g transpose are

$$\mathrm{mul}_n(f,g) = C_n D_{\mathcal{R}}(A_B(I_n f) \cdot A_B(I_n g)), \tag{20}$$

$$(\partial_f \mathrm{mul}_n)^{\mathsf{T}} z = I_n^{\mathsf{T}} A_B^{\mathsf{T}} M_{A_B(I_n g)}^{\mathsf{T}} D_{\mathcal{R}}^{\mathsf{T}} C_n^{\mathsf{T}} z. \tag{21}$$

The transpose of selection in C_n is explicit zero insertion; the transpose of the packing is the reversed physical permutation.

Operation	Charged bit time
Affine evaluator or ordinary transpose	$O(bDK)$
Pointwise products, weights and scalar transposes	$O(N_K K^\rho)$
Native field Grams and root actions	$O(bD)$
Input/output representation changes	$O(N_K K^\gamma)$
Public top-level setup	$\mathrm{poly}(K)$

Every recursive layout is restored to native words before descent; the local rectangular padding is not multiplied across dimensions. A fixed number of subroutine tape banks and local depth-first parking stacks suffices; no descendant crosses a parked ancestor. Cleanup scans only the current used intervals. The first line of the table is $O(N_K K) = O(n \log n)$, and the other lines fit within it. This proves Theorem 2.4 for both operand branches.

7 A uniform fixed-tape bound for the binary tensor transform

Write

$$H_2 = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}, \qquad V = 2^t D.$$

The purpose of this section is to give the machine-level tensor lemma used in the reduction. We state explicitly the one imported movement theorem. All constants below, including the number of tapes, may be enormous, but are independent of both t and D.

Contract 7.1 (Imported equal-chunk interchange)**.** *We use only Lemma 12 of [7]: on a fixed finite-alphabet, fixed-number-of-tapes machine, an arbitrary binary array of shape*

$$[P] \times [2^u] \times [G] \times [2^u] \times [B]$$

can have its two u-bit address fields interchanged in $O(V_0 u^\tau)$ bit steps, for one fixed $\tau < 1$, where $V_0 = PGB2^{2u}$ is the entire stored bit volume. Prefix, gap, suffix and attached payload

are arbitrary; $B = 1$ is allowed. The source theorem includes parameter processing, padding, unpadding and workspace cleanup. We do not use its later integer-multiplication theorem. Every use of this contract below moves a bounded number of equal-width chunks and charges the full attached record volume.

7.1 Opaque records and packed address shears

A routing array has M address records and an opaque suffix of R payload bits per record. Let

$$V = MR, \qquad p = \lceil \log_2(2M) \rceil.$$

The p-bit key contains every field consulted by the routing function, including physical bit planes, dirty selector/tile fields, and role rows. No payload bit is an address control. We require R to dominate every fixed polynomial in p in the large-height regime. This condition is later verified from the four-block layout, not assumed uniformly for small heights.

Lemma 7.2 (Opaque-record packed shear)**.** *Let T be a contiguous u-bit address field and W a disjoint dirty u-bit tile field. Suppose g is a fixed polynomial-time function of address controls disjoint from T, W, with u output bits. Controls are presented before the active target in tape order; if necessary, a bounded number of width-u chunk interchanges docks the target after them. The tile is docked as needed for each rotation. Full control fields need not move if a consulted prefix suffices; full-key evaluation is charged below. Then*

$$(T, W, \text{controls}, z) \longmapsto (T \oplus g(\text{controls}), W, \text{controls}, z)$$

is an exact permutation of complete records and their attached payload. For fixed $\tau < 1$ from Contract 7.1, its cost is

$$O\Big(V(u^\tau + 1)\log^2(p+2) + Mp^C\Big) \tag{7.2.1}$$

for a fixed C, provided $u \le p$ and R dominates p^C. The ordinary binary transpose is its inverse and has the same bound.

Proof. Here are the record-level charges needed beyond the imported interchange. A record can be copied in $O(R)$ movement: keep a binary countdown initialized to R, scan the opaque payload once, and decrement a low-end counter per bit. The total trailing-zero traversal over the countdown $R, R-1, \dots, 1$ is $O(R)$; resetting costs $O(\log R) \le O(R)$. No payload value is interpreted, and no R-bit displacement is formed.

For a fixed control fiber, every target value occupies a consecutive block of complete records. If the suffix contains B_f records per target value, the rotation cut is at δB_f records. The offset δ and cut count are computed from the p-bit address in $p^{O(1)}$ steps per fiber; the two pieces are copied by the record counter. Across fibers the movement is $O(V)$ and metadata is $O(Mp^C)$. This proves the controlled-rotation bound without requiring that a long payload be a unit-cost memory cell.

For completeness, the exact packed identity is as follows. For integers a, w and fixed Boolean control c, apply the following updates chronologically; each parity test uses the current values:

$$\begin{aligned}
a &\leftarrow a - c[w \equiv 0 \pmod 2] && (1)\\
w &\leftarrow w - c[a \equiv 0 \pmod 2] && (2)\\
a &\leftarrow a + c[w \equiv 0 \pmod 2] && (3)\\
w &\leftarrow w + c[a \equiv 0 \pmod 2] && (4)\\
w &\leftarrow w + c[a \equiv 1 \pmod 2] && (5)\\
a &\leftarrow a + c[w \equiv 1 \pmod 2] && (6)\\
w &\leftarrow w - c[a \equiv 1 \pmod 2] && (7)\\
a &\leftarrow a - c[w \equiv 1 \pmod 2] && (8)
\end{aligned}$$

Set $\epsilon(x) = 1 - 2(x \bmod 2)$. Their composition is

$$(a, w) \mapsto (a + c\epsilon(a), w). \tag{7.2.2}$$

For $c = 1$, direct substitution in the eight current-value predicates gives the following four cases, where the entries are total displacements from the initial pair:

$(a \bmod 2, w \bmod 2)$	Δa	Δw
$(0, 0)$	$+1$	0
$(0, 1)$	$+1$	0
$(1, 0)$	-1	0
$(1, 1)$	-1	0

For $c = 0$ both displacements are zero. Each update predicate depends only on parity, so translating the initial pair by $(2r, 2s)$ translates every intermediate pair by the same amount and leaves all eight decisions unchanged. The four cases therefore prove for all integers (including negative values)

$$(a, w) \longmapsto (a + c(1 - 2(a \bmod 2)), w),$$

which flips the selected bit, preserves the quotient of a by two, and restores the arbitrary auxiliary w. Substituting $c = g_i(\text{controls})$ is valid because the address controls used to compute g_i are frozen throughout the word; current target/tile parities are intentionally updated and tested as displayed.

Put $K = \lceil (\log_2(p + 2))^2 \rceil$, with a fixed lower cutoff, and partition target positions into K residue classes. In each class temporarily omit its highest selected bit and treat it by one direct controlled-bit route. Every other selected bit starts a real K-bit lane inside the actual u-bit field; its remaining $K - 1$ bits are guards. There is no extension of the address range to a multiple of K. An integer lane is safe when its quotient lies in $[10, 2^{K-1} - 11]$. The eight rotations move its segment by at most the fixed margin, so (7.2.2) acts as the desired independent bit toggle without carry into a neighboring lane. At most $80u2^{-K}M$ records lie outside the safe guard set.

Let π be the packed permutation and σ the desired XOR. Its controls are frozen and σ changes only selected low bits, so the guard-bad set is invariant under σ. On its complement $\pi = \sigma$; bijection therefore also makes the bad set invariant under π. For a current bad-record key y, its exact destination is $\sigma(\pi^{-1}(y))$. Compute π^{-1} by replaying the eight rotations backward on the key, reversing each sign and testing the current key at that step. The full controls used by g are unchanged, so this takes polynomial time in the p-bit key and never inspects the payload.

Extract bad records sequentially to a work stream with their complete destination keys, and mark the holes in the full array. Least-significant-bit radix sorting uses two bucket tapes and alternating pass tapes: split on the current destination bit, preserving order within each bucket, and concatenate. Every pass moves each entire key and attached payload, including return scans. Merge the sorted records into the marked holes in increasing full-address order. The destination keys are distinct and fill exactly the holes by the invariance and bijection just proved. If b_{bad} is the bad-record count, this costs $O(V + Mp^C + b_{\text{bad}}p(R + p))$, hence

$$O\Big(V + Mp^C + u2^{-K}Mp(R + p)\Big).$$

The last term is $O(V)$ since $up2^{-K} = o(1)$ and $R \gg p^C$. Each tile rotation docks the current target or tile chunk after all controls by a constant number of width-u interchanges from Contract 7.1; arbitrary gaps and payload remain spectators. Summing the K classes gives (7.2.1). The record-counter, full-key and docking constructions above supply the movement and ordering charges. □

The construction has a precise control-order condition: at every rotation, the frozen address controls precede the current target and are disjoint from target and tile. In the physical order $[X_h][Y_h][Q_h][Z_h][W_h]$, the full X, Y mask controls precede the Q, Z oracle targets. For uniform bit-plane shears, only consulted prefixes move: a bounded number of width-u swaps docks the target behind its controls and tile. For example, the middle shear of the controlled swap reads updated X and uses $[Z, X, Q, W, Y]$. The remaining suffix bits stay fixed as address context. Thus a recursive call with $u \ll h$ never swaps an h-bit mask field. Full-key selector calculation is charged in Mp^C; the source's narrower controlled-shift lemma is not invoked with an h-bit control pretending to have width u.

7.2 Dirty active-rank selection

Represent a base-four digit as $d_i = 2X_i + Y_i$, with X, Y bit planes. Digit 3 is idle and the active mask is

$$S(d) = \{i : d_i < 3\}.$$

For a transposition τ of two active symbols that fixes the third active symbol and idle symbol 3, let T_Z apply τ at positions where a dirty selector Z_i is one. Each completed T_Z preserves S, even though its binary implementation can temporarily change it.

Lemma 7.3 (Dirty selector cancellation and SUM routing)**.** *For any mask-only selector $J(S)$, the word*

$$Z\oplus = J(S); \quad T_Z; \quad Z\oplus = J(S); \quad T_Z \tag{7.3.1}$$

restores every initial dirty Z and equals T_J. Replacing J by $J \wedge Q$ works for arbitrary fixed dirty Q. If $J(S)$ specifies disjoint matched active source and target ranks and F_a marks a target whose paired source has current symbol a, then

$$Q\oplus = F_a; \quad L_Q; \quad Q\oplus = F_a; \quad L_Q, \qquad L_Q = T_{J\wedge Q}, \tag{7.3.2}$$

restores Q, Z, W and implements the source-value-conditioned target transposition. A fixed sequence gives target $\leftarrow$ target + source modulo three.

Proof. Every completed T_Z preserves the active mask S, so the mask-only selector $J(S)$ is invariant at the completed-operation boundaries. If the original value of Z is z, the two transpositions in (7.3.1) compose to $T_{z\oplus J}T_z = T_J$ and the second selector update restores z. The same calculation works for $J \wedge Q$, because Q is fixed throughout this inner word.

In (7.3.2), J selects the target ranks of the disjoint matched pairs. Each completed $L_Q = T_{J\wedge Q}$ therefore changes only selected targets and preserves their paired-source symbols. The mask and those source symbols determine F_a, so F_a is unchanged at the *completed L_Q boundaries* where it is recomputed. Source symbols may change during an inner T_Z or its binary shears; the outer selector queries occur only after the whole L_Q is complete. Applying the same involution cancellation to the two Q updates restores its arbitrary initial value and leaves $T_{J\wedge F_a}$. Each inner word restores Z, and the packed-shear realization below restores the tile field W. This proves the stated outer cancellation for all dirty values.

The controlled transpositions have literal binary shears

$$\begin{aligned} &(0\ 1): \quad Y\oplus = Z \wedge \neg X, \\ &(1\ 2): \quad X\oplus = Z \wedge Y; \quad Y\oplus = Z \wedge X; \quad X\oplus = Z \wedge Y. \end{aligned} \tag{7.3.3}$$

The second acts successively as $(x, y) \mapsto (x \oplus y, y) \mapsto (x \oplus y, x) \mapsto (y, x)$ when $Z_i = 1$. Each displayed shear is an instance of Lemma 7.2. For the middle shear, the control is the updated X, not the original one. Its required order is $[Z, X, Q, W, Y]$ after docking the current target; the other two shears use $[Y, Z, Q, W, X]$ and $[X, Z, Q, W, Y]$ respectively, with unused fields

as spectators. Restore the physical order after each elementary shear. Chronological (1 2) then (0 1) cycles $0 \to 1 \to 2 \to 0$; reversing the order gives its inverse. Conditioning these orientations on paired-source symbols 1 and 2 realizes addition modulo three. A coordinate swap is three SUMs plus a separate negation (1 2); no false three-shear swap identity is used. □

All controls in the table below are address fields and exclude the elementary target and tile field.

update	controls	docked order
$Q \oplus = F_a(X, Y)$	X, Y	$X, Y \mid Q \mid W$
$Z \oplus = J(S) \wedge Q$	X, Y, Q	$X, Y, Q \mid Z \mid W$
$Y \oplus = Z \wedge \neg X$	X, Z	$X, Z \mid Y \mid W$
$X \oplus = Z \wedge Y$	Y, Z	$Y, Z \mid X \mid W$
$Y \oplus = Z \wedge X$	X, Z	$X, Z \mid Y \mid W$

The '|' notation marks the rightmost dock; each target/tile exchange has width u. For Q, Z oracles the full X, Y controls precede the targets. For uniform X/Y gates, dock the target prefix past the other consulted prefixes with a fixed number of width-u swaps; the middle Y shear in the controlled swap reads updated X and uses the order $[Z, X, Q, W, Y]$. Unselected suffix bits remain fixed address context. Selector evaluation, including full-mask active-rank scans and paired-source lookup, is polynomial in the address-key width and included in Mp^C.

7.3 Prefix-local calls and exact exceptional correction

A fixed frame node of width e needs only the first e active ranks. Set

$$u = \min\{h, \lceil C_0(e + \log_2(p+2))\rceil\} \tag{7.1}$$

for a fixed sufficiently large C_0. All physical targets and the Q, Z, W work fields use their first u positions; full-mask fields can be read as controls. When the pass is eligible only for $|S| \geq e$, its selector is set to zero if the prefix has fewer than e active positions. If $u < h$, then $u \geq 2e$ (increasing the fixed lower cutoff if necessary). Under the uniform full address array, the prefix indicators are independent Bernoulli(3/4), so

$$\Pr\{|S \cap [u]| < e\} \leq e^{-u/24}. \tag{7.2}$$

If $u = h$, eligibility makes the bad set empty.

The ideal matching acts only on active symbols and preserves S. The implemented completed shears also preserve S. Thus the prefix-bad set is invariant. If the local packed construction has its separate guard exception, that set is already corrected by Lemma 7.2; it is not confused with the prefix exception. For the latter, extract every bad record, compute the exact full destination key in polynomial p work, radix-sort distinct destination keys, and reinsert at marked holes. Its full cost is

$$O\Big(V + Mp^C + e^{-u/24} Mp(R+p)\Big). \tag{7.3}$$

Choose one fixed C covering all address-polynomial degrees in the finite collection of routing algorithms, and take $C_0 \geq 240(C+10)$. The role recursion below has $O(h^\sigma)$ volume-weighted nodes, where $\sigma < 1$. At every node $p = \Theta(h)$ because all physical-mask and dirty fields remain present. The prefix exception fraction is at most $(p+2)^{-C_0/(24\log 2)}$. Summing its sort cost over the nodes and the polylogarithmically many cardinality groups is bounded by

$$O\Big(Vh^\sigma \operatorname{polylog}(h+2)(p+2)^{C+2-C_0/(24\log 2)}\Big) = O(V).$$

The metadata term is separately absorbed by the exponentially wide payload. Role splitting is independent of the mask, so the bound holds in every child. No mask or payload is dropped. Combining (7.2.1)–(7.3), one node has normalized movement toll

$$O\big(\mathrm{polylog}(p+2)\big((e+\log(p+2))^{\tau}+\mathrm{polylog}(p+2)\big)\big)\,, \tag{7.4}$$

when $Mp^{C}=O(V)$. In particular this pays for small recursive prefixes instead of charging h^{τ} at every node.

7.4 Ternary phase frame and fixed role recursion

Let $K=\mathbb{F}_4=\mathbb{F}_2[\eta]/(\eta^2+\eta+1)$. The address field $\mathbb{F}_3$ is distinct from the scalar field K. Let F_3 denote the three-point Fourier matrix over K, normalized by $1/3=1$, and put

$$C=F_3^{-1}\,\mathrm{diag}(1,\eta,\eta)F_3=\eta I_3+\eta^2 J_3=\begin{pmatrix}1&\eta^2&\eta^2\\ \eta^2&1&\eta^2\\ \eta^2&\eta^2&1\end{pmatrix},\qquad C^3=I.$$

For a nondegenerate subspace $U\subseteq\mathbb{F}_3^h$, Fourier conjugation of the phase $x\mapsto\eta^{(P_Ux)\cdot(P_Ux)}$ defines C_U. Orthogonal direct sums multiply phases. On a comparable geometric transition from U to V, write $P_V-P_U=\epsilon P_R$: if $U\subseteq V$, then $R=V\cap U^{\perp}$ and $\epsilon=1$; if $V\subseteq U$, then $R=U\cap V^{\perp}$ and $\epsilon=-1$. For a nondegenerate residual direction v of original norm $\nu=v\cdot v\in\mathbb{F}_3^{\times}$, its phase parameter is $\lambda=\epsilon/\nu\in\mathbb{F}_3^{\times}$: its Fourier phase is $\eta^{\lambda(v\cdot x)^2}$. The norm is taken before address placement; the placement need not be an isometry. The same signed-direction rule applies to the additional source-frame correction in Appendix A. The corresponding coordinate kernel is C or C^{-1} according to λ. For a fixed node edge, each basis direction $v_j\otimes I_f$ has the same signed type $\lambda_j=\epsilon/\nu_j$ across its f coordinate copies. The recursion therefore has one width-f child for each signed fixed direction; mixed signed types are separate rank-counted children, not one child whose type varies over an arbitrary selected subset. Since coefficient conjugation $a\mapsto a^2$ on the two explicit $\mathbb{F}_4$ bit planes takes C to C^{-1}, each inverse-phase child is one forward child plus fixed linear work. This does not assume Frobenius is free in a growing field. Fixed $\mathbb{F}_{16}$ payload lanes are represented by their two $\mathbb{F}_4$ coefficient planes.

Here is the finite motif defining the role network. Put $H=100$ and let vertices be triples $T\in\binom{[H]}{3}$, adjacent when $|S\cap T|=1$. Write

$$v=\binom{H}{3},\quad z=3\binom{H-3}{2},\quad N=v^3,\quad I=3v^2,\quad m_0=(H+1)^3,$$

$$W=2N+I(vz+H),\quad L=IH(H+1),\quad s=Wm_0-N+2L. \tag{7.5}$$

For the lifted ternary frame use $\mathbb{F}_3^{H+1}$ with Gram form $\mathrm{diag}(2I_H,1)$ and line label $t_T=(1_T,1)$. Its norm is $2\cdot3+1=1$ in $\mathbb{F}_3$, and adjacent lines are orthogonal since $2|S\cap T|+1=0$. The form is isometric to the ordinary dot product: on each coordinate pair use $(a,b)\mapsto(2a+2b,2a+b)$, leaving the last coordinate fixed. The role labels in the scalar copy/gather/scatter schedule are tensor products of the common line and its orthogonal complements. Appendix A gives the graph, gate labels, signed rank telescope and endpoint correction. Appendix A also proves the scalar schedule and its fixed-tape array compiler, following Sections 3–4 of [7]. Contract 7.1 is the external movement theorem. The local ternary phases and finite counts use the explicit data displayed here.

The role count has two data banks, $2N$ data roles, and $I(vz+H)$ side/center roles. Only the H central wires per invocation lose rank, each by $H+1$, giving total loss L. The signed dimension telescope plus twice the downward variation gives edge-rank sum $Wm_0-2N+2L$.

Use the source-frame convention of Appendix A throughout. On each exceptional $X_a \to Y_a$ route, choose source frame C_U^{-1} and sink frame $C_{U^\perp}$. The endpoint is

$$C_{U^\perp}(C_U^{-1})^{-1} = C_{U^\perp}C_U = C_{\text{full}}.$$

The first edge has head frame C_U; its actual operator is $C_U(C_U^{-1})^{-1} = C_U^2 = C_U^{-1}$. This is one rank-one child beyond the geometric variation, on each of the N exceptional routes, giving the $+N$ in the child count. On scratch routes the source label is 0 and the sink label is the full space, so their endpoint is already C_{full}; they are arbitrary dirty data and are retained. Thus the corrected child count is s in (7.5): exceptional Y sinks carry $U^\perp$, while scratch sinks carry the full space, and these cases are not conflated.

$$\begin{aligned} M_c &= m_0 = 1{,}030{,}301, \\ W &= 177{,}176{,}569{,}091{,}445{,}000{,}000, \\ s &= 182{,}545{,}196{,}308{,}841{,}424{,}366{,}000{,}000. \end{aligned}$$

$$1 - \frac{s}{WM_c} = \frac{337}{23271753837835} > 20 \cdot 2^{-50}, \qquad \sigma = \log_{M_c}(s/W) < 1 - 2^{-50}. \tag{7.6}$$

Appendix A derives these identities from the graph and gate-label dimensions, including the source/sink endpoint correction. Bounded checks corroborate the finite arithmetic but are not used as proof. The scalar schedule, common-frame identity, and fixed-tape role compiler are proved in Appendix A, with attribution to [7].

At a node $e = M_c f$, the fixed frame change on M_c slots is a constant matrix tensored with I_f. Factor it into a fixed number of elementary row-addition batches over $\mathbb{F}_3$; each batch has disjoint source and target slots, and is implemented by Lemma 7.3. Roles are split independently of masks. Every child has volume V/W and still has the full physical mask distribution. Parent gates and parking/restoration cost $O(V)$ per child, but the number s of children is a fixed machine constant. With the prefix toll, the normalized recurrence is

$$\begin{aligned} H(e;h) \le \frac{s}{W} H(\lceil e/M_c \rceil; h) \\ + O\big(\text{polylog}(p+2)\big((e + \log(p+2))^\tau + \text{polylog}(p+2)\big)\big)\,. \end{aligned} \tag{7.7}$$

For every fixed $\lambda > \max(\tau, \sigma)$, $\lambda < 1$, this is $O(e^\lambda \,\text{polylog}(p+2))$. At the base $e = 1$, the selected-rank kernel is the circulant $I + \eta^2(X + X^{-1})$, where X increments that active trit. Implement X and X^{-1} by mask-only transposition selectors restricted to a prefix of $O(1 + \log p)$ positions; repair exceptional prefix masks exactly, retain a fixed number of input copies, and perform fixed-field additions. This costs $O(V \,\text{polylog}(p+2))$, not an h-position scan per data bit. A bounded set of widths below M_c uses a fixed number of these prefix-local operations.

To cover every mask cardinality $a = |S|$, express a in base M_c and process its consecutive active-rank intervals of power widths. These intervals partition the active ranks for each mask, and their widths sum geometrically. To place an interval at the front, express its mask-dependent coordinate permutation as two involutions; each involution is a disjoint matching of active-coordinate swaps implemented by the priced SUM/negation gadgets. Placement and its inverse have full-width cost $O(Vh^\tau \,\text{polylog}\, p)$ per group. There are $O(\log h)$ groups, so their total is $O(Vh^\rho \,\text{polylog}\, h)$ for some fixed $\rho < 1$. No low-active mask is discarded.

One-time row padding. Let $k = \lceil \log_{M_c} h \rceil$ and choose $r = \lceil k \log_2 W \rceil$ existing spectator index bits for the row field. Then $2^r \ge W^k$ and $r = O(\log h)$. Pad its range once to $L_0 = W^k \lceil 2^r / W^k \rceil$, so $2^r \le L_0 < 2^{r+1}$. The compiler of Appendix A splits $q = Wg + (w-1)$; at depth j a child has L_0/W^{j+1} rows, divisible by W^{k-j-1}. Every child retains the full

physical-mask and selector fields independently of this split. Intermediate gates may make artificial rows nonzero; no intermediate zero promise or crop is used. The completed endpoint identity acts independently on every original row, including arbitrary dirty scratch roles. Only that full boundary permits cropping artificial rows. The child factor remains s/W, without a recursive padding multiplier.

Mask-dependent group eligibility. A power-width group has enable predicate $E_{j,\kappa}(S) = [a_j(S) \geq \kappa]$, where $a_j(S)$ is the jth base-M_c digit of $|S|$ and $1 \leq \kappa < M_c$. Include it in all dirty selector oracles, including placement and its inverse, and choose all frames to be the identity on disabled sectors. The predicate is inherited by recursive children and is unchanged by completed matching maps. The common-frame identity then gives identity on disabled sectors after the role permutation is undone; the same fixed child count is an upper bound. At a leaf use the conditionally selected trit increment X_E. The kernel is $I+\eta^2(X_E+X_E^{-1})$, which equals I on a disabled sector because $X_E = I$ and the scalar characteristic is two. Predicate calculation is polynomial in p and is included in the metadata toll.

7.5 Internal payload and four-block layout

For clarity about the fixed node factorization used above, a chosen basis direction can be negated to make its completed basis matrix have determinant one. This does not change the directional operator, whose kernel uses both v and $-v$. A determinant-one matrix over $\mathbb{F}_3$ is a product of elementary row additions. Thus this step does not silently factor an arbitrary determinant-minus-one matrix into transvections; an explicit active-symbol negation could also be charged by the already priced selector gadget.

Write $t = 8h + t_0$ with $0 \leq t_0 < 8$. Handle the fewer than eight leftover binary axes by a bounded direct circuit. For the other $8h$ axes, reserve four existing h-digit base-four blocks, or $8h$ binary index bits. In a selected block, X, Y occupy its $2h$ bits. Three other blocks supply distinct Q, Z, W fields of h bits each. An independent r-bit row field, $r = O(\log h)$, is reserved from the remaining indices; the remaining spectator index bits and the external coefficient word form the contiguous opaque suffix. Counting all nonpayload address fields gives

$$M = 2^{5h+r}, \qquad R = D\,2^{3h-r}, \qquad p = \lceil \log_2(2M) \rceil = O(h), \qquad MR = D2^{8h}. \tag{7.8}$$

The displayed counts precede row padding. Afterwards

$$M' = L_0\,2^{5h}, \qquad R' = R, \qquad M \leq M' < 2M, \qquad p' = \lceil \log_2(2M') \rceil = O(h).$$

The row is an explicit Cartesian shape coordinate of range L_0, which need not be a power of two. Use the mixed-radix ordinal of the actual full address as the sorting key; it is below M' and is computed in polynomial time in p'. One additional metadata bit encodes the padded range; no new binary data axis is appended. Equal-chunk interchange permits this arbitrary row prefix. At depth j replace L_0 by L_0/W^j; the full X, Y, Q, Z, W fields remain, so $p_j = \Theta(h)$ and R is unchanged. For sufficiently large h, $r \leq 2h$, and thus $R \geq D2^h$. The finite remaining heights use direct circuits.

A fixed factor accounts for encoding binary payload lanes in $\mathbb{F}_{16}$. Since $r = O(\log h)$, R exceeds every fixed polynomial in p for all sufficiently large h, uniformly for $D \geq 1$. The remaining finite set of heights uses direct finite-width scans, whose constant is independent of D. Record formation is a radix split of the original column address; no gather or transpose is implicit. The three borrowed blocks and all dirty routing fields are restored after the call and can be reused in later block calls. No bit is both row address and opaque payload or both a router field and role field.

7.6 A fixed positive word and completion of the tensor theorem

The physical layout changes in the preceding construction have the following cost. Regard the $8h$ original address bits as eight contiguous h-bit planes. Equal-width plane interchanges promote the selected pair to X, Y and place three spectator planes at Q, Z, W. A fixed number of Contract 7.1 calls costs $O(Vh^{\tau})$. Move the $r = O(\log h)$ role bits to the front by individual digit moves, costing $O(V \log h)$. The resulting layout is

$$[\text{role}][X_h][Y_h][Q_h][Z_h][W_h][\text{opaque suffix}].$$

After the completed block map, crop only the full padded row invocation and undo these moves before promoting the next block. Pairing the two bitplanes at equal positions is a logical pairing of identical H_2 tensor axes: the target tensor is invariant under reordering those identical factors. The routing algorithms act directly on these bitplanes, so no uncharged interleaving/perfect shuffle is used.

Let $\eta = t^2 + t$ in $E = \mathbb{F}_2[t]/(t^4 + t + 1) = \mathbb{F}_{16}$. The finite-word appendix, labelled `app:word`, proves that $H_2 \otimes H_2$ is a fixed word with 384 forward letters $A = C \oplus [1]$ and 392 invertible monomial letters over E. Its projector/commutator derivation gives four elementary matrices $U_{ij}(1)$ using 96 forward A calls each, then the two wire permutations; it is an algebraic identity, not an existence/search claim. Applying the word tensorwise over h digits reduces its A letters to the role algorithm above. A monomial letter's local permutation is in $S_4 = \mathrm{AGL}(2, \mathbb{F}_2)$, hence a fixed NOT/CNOT address circuit. It can change the active mask, so it is completed as a full address permutation and is never commuted through a mask-preserving frame; the next A call recomputes its mask selectors on the resulting complete address array. Its diagonal contributes a product of h fixed E scalars, computed from the p-bit key in $O(h)$ work per record and applied to the fixed-degree payload in $O(R)$ per record. These charges are $O(Vh^{\tau} \operatorname{polylog} h)$ for permutations and $O(V)$ for diagonal actions. The word length and number of block/layout interchanges are fixed.

Encode each binary payload bit as a 0/1 coordinate in a fixed E lane, padding by a factor at most four. Each E scalar is a fixed four-bit binary map. The complete word matrix has entries in $\mathbb{F}_2$ and equals the target, so the padded lanes return to the binary subspace and the same transform is applied independently to all D bits. The ordinary binary transpose is the same transform because $H_2^{\otimes t}$ is symmetric; its inverse is two forward applications since $H_2^3 = I$.

Theorem 7.4 (Uniform fixed-tape H2 tensor bound)**.** *Using the equal-chunk interchange theorem of [7, Lemma 12], there is one deterministic fixed-alphabet multitape machine, with a fixed finite number of tapes, which for every $t, D \geq 1$ computes*

$$H_2^{\otimes t} \otimes I_D$$

on 2^t records of D bits in $O(2^t D\, t^{\beta})$ steps for some fixed $\beta < 1$. The same bound holds for its ordinary binary transpose; its inverse has the same bound up to a constant factor.

Proof. Choose $\lambda > \max(\tau, \sigma)$ below one. The role recurrence and geometric grouping give each A tensor call $O(Vh^{\lambda} \operatorname{polylog} h)$; top-level placement, packed shears, and layout changes have strict sublinear height exponents as well. A fixed number of calls per positive-word letter and four blocks changes only constants. Monomial permutation and diagonal costs were charged above. Absorb the fixed polylogarithm into a slightly larger exponent $\beta < 1$. The internal layout (7.8) makes $Mp^C = O(V)$ and the exceptional sorts fit uniformly for arbitrary D. Bounded t is handled directly. This proves the claimed bound from the cited interchange theorem and the local routing, role, and finite-word proofs in this section and its appendices. Symmetry gives the binary transpose; $H_2^3 = I$ gives the inverse. □

Remark 7.5 (Dependency and arithmetic scope)**.** The external machine input is Lemma 12 of [7]. The scalar bank exchange, array-frame invariant, local fixed-tape role schedule, ternary labels, and finite word are proved here. The input/output transform is binary; its internal scalar alphabet is the fixed field $\mathbb{F}_{16}$. The auxiliary growing coefficient fields of the multiplication reduction are opaque payload for this theorem. The $\mathbb{F}_{16}$ lanes encode bits; no embedding of $\mathbb{F}_{16}$ into those growing fields is required. No ordinary complex Walsh–Hadamard bound follows from this binary statement: the binary matrix H_2 is a different invertible kernel. The finite witnesses in Section 8 corroborate finite identities and are not used to infer the asymptotic bound.

Corollary 7.6 (Exact multiplication consequence)**.** *Using [7, Lemma 12], Theorem 7.4 supplies the tensor-transform premise of Theorem 2.4. Hence the stated fixed-tape model admits an exact $O(n \log n)$ algorithm for multiplying degree-$< n$ polynomials over $\mathbb{F}_2$. For either fixed operand, the ordinary binary transpose has the same bound.*

Corollary 7.7 (Fixed characteristic-two extensions)**.** *For every fixed positive integer s, the same bit bound holds over $\mathbb{F}_{2^s}$ in any fixed polynomial-basis representation.*

Proof. Write coefficients in a fixed s-element binary basis. A product of two polynomial coefficient lists uses s^2 binary polynomial products, followed by the fixed field's structure-constant linear maps. The number of products is constant and the latter maps cost $O(n)$. For a fixed operand, reverse these explicit constant-degree coordinate maps and use the binary fixed-operand transposes. Thus the same bound holds for the ordinary binary adjoint as well. The field and its representation are fixed independently of n. □

8 Exact computational witnesses and reproducibility

The companion programs are bounded C++17 algebra checks. Every executable installs a Windows job limit of 64 MiB process memory and 10 process CPU seconds before starting algebra, and refuses to continue if the limit installation fails. Fixed arrays bound every instance. The checks use neither a growing dense transform matrix nor an enumeration of the enormous role graph. Compilation is separate from these runtime limits.

The evaluator witness uses the ordinary pairwise Pascal core on its small instances. It independently compares the resulting full map with Horner evaluation, the radix extraction with monic polynomial division, and the actual transpose with literal binary Vandermonde columns. Its local Karatsuba/module/Gram implementation is compared against an independent bit-serial ambient field implementation. Both radix roundtrips and the distinct inverse transpose are checked.

The full product witness composes native input embedding, affine shift, the new prefix evaluator, pointwise multiplication, the weights (15), and the decoder (16). It compares against a separate schoolbook carryless product and reduction. Arbitrary ambient covectors exercise evaluator and decoder transposes and the complete fixed-operand product adjoint; tests do not restrict those covectors to primal images or to smaller subfields.

Witness	Fixed scope	Assertions
Prefix evaluator	$K = 1, \ldots, 5$, $b \le 1024$, E of 54 bits	1,481,563
Parent and raw product	$K = 1, \ldots, 4$, parent width ≤ 4608 bits, raw inputs ≤ 2304 bits	784,218

The saved reports in `witnesses/code/` were produced by the bounded programs in the reviewed Pascal/Taylor source packet on 30 October 2026. They are historical receipts copied

with their exact sources, not runs made for this focused release. The first covers $K = 1, \ldots, 5$ and records 1,481,563 assertions, including 47,376 local/full-field phase comparisons. The second covers $K = 1, \ldots, 4$ and records 784,218 assertions. The included `witnesses/verify-witnesses.ps1` script can reproduce these finite checks under the stated per-process guard. The witnesses use ordinary finite Pascal cores; they neither execute nor certify the asymptotic fast-H2 machine or the $O(n \log n)$ theorem. For source provenance, the inspected local text snapshot of [7] has SHA256

```
BD480BB7A3F7B662B99EEBEBDBFDB01CCA8F
AB9D3BB46F5CBD0DCD9E0C207493
```

where the two lines are concatenated. This identifies the historical local text snapshot. The bibliography separately pins the citation revision to `4eaee87548250b2682affda7d05d947c05554bae`; the text hash is not a claim of byte identity with the raw TeX source. Neither record turns the source theorem into a formally verified theorem. No approximate complex arithmetic, Gaussian resampling, least-prime hypothesis or data-dependent finite-field search is used by the construction described here.

A Finite role and phase certificate

This appendix records the finite network used in the ternary phase construction. It separates three ingredients: an exact scalar bank exchange with dirty scratch, a subspace-label assignment to every wire, and the endpoint correction that makes the induced phase the same on data and scratch. All role counts below count physical wires with multiplicity; equal labels on distinct wires are never identified. The label argument is the finite-dimensional certificate underlying the role recurrence, not by itself a proof of its tape implementation. The scalar schedule and array-frame construction follow Sections 3–4 of [7]; their needed algebraic and fixed-tape arguments are given below. The ternary labels, endpoint choices and directional-child counts are explicit.

A.1 The scalar eight-gate schedule

Fix an integer $h \geq 6$, and let $\mathcal{T} = \binom{[h]}{3}$ be the set of triples. Put $v = |\mathcal{T}| = \binom{h}{3}$. Declare distinct $S, T \in \mathcal{T}$ adjacent when $|S \cap T| = 1$. Each S has

$$z = 3\binom{h-3}{2}$$

ordered neighbors: choose the common element in three ways, then choose the two elements of T outside S.

An invocation has data banks $x = (x_T)_{T \in \mathcal{T}}$ and $y = (y_S)_{S \in \mathcal{T}}$. It also has an arbitrary side-scratch value $a_{S,T}$ for each ordered adjacent pair and an arbitrary central-scratch vector $c = (c_i)_{i \in [h]}$. Define the incidence maps over $\mathbb{F}_2$ by

$$V_{(S,T),T'} = [T = T'][|S \cap T| = 1], \quad J_{S,(S,T)} = [|S \cap T| = 1], \quad G_{i,T} = [i \in T], \quad R_{S,i} = [i \in S].$$

Thus V copies the indicated source coordinate to the side role, J gathers side roles to a target, G gathers data to the central roles, and R scatters central values to targets. Execute the following eight updates chronologically; scratch named in an update is read as a control and

is not changed by that update:

0	$y \leftarrow y - Ja$
1	$y \leftarrow y - Rc$
2	$a \leftarrow a + Vx$
3	$c \leftarrow c + Gx$
4	$y \leftarrow y + Rc$
5	$y \leftarrow y + Ja$
6	$c \leftarrow c - Gx$
7	$a \leftarrow a - Vx.$

If the initial scratch is (a_0, c_0), its final value is again (a_0, c_0). The net target increment is

$$-Ja_0 - Rc_0 + R(c_0 + Gx) + J(a_0 + Vx) = (RG + JV)x.$$

For any S, T, $(RG)_{S,T} = |S \cap T| \pmod 2$, while $(JV)_{S,T}$ is one exactly for intersection size one. If $S = T$, the intersection size is three, so $RG = 1$ and there is no side role. For distinct triples, intersection sizes are $0, 1, 2$: at size one both terms are one and cancel, and at sizes zero or two both vanish. Hence $RG + JV = I$. This proves that the invocation performs $y \leftarrow y + x$ for every starting value of every scratch register.

Three disjoint-scratch stages give a bank exchange. Stage one applies the forward schedule from X to Y. Stage two applies its exact inverse from Y to X, and stage three again applies it from X to Y. On arbitrary initial banks the states are

$$(x, y) \longmapsto (x, y + x) \longmapsto (-y, y + x) \longmapsto (-y, x).$$

Over $\mathbb{F}_2$, minus equals plus, so the final state is (y, x). Every invocation scratch is restored before the next stage, and the stages use distinct scratch roles. Thus the network is an exact permutation of data roles and identity on scratch roles, including when the scratch was dirty initially.

Let $N = v^3$ be the number of paired data coordinates in the three-factor role array and let $I = 3v^2$ be the number of invocations in its three stages. Each invocation has vz ordered side roles and h center roles. Including two data banks, the full role count is

$$W = 2N + I(vz + h).$$

This count is used as the arity of the finite role construction.

A.2 Array frames and a fixed-tape compiler

Replace each scalar role by an array on the same ordered finite address set Ω, and apply the scalar gates pointwise. At each source, gate and sink v, assign an invertible K-linear operator D_v on K^Ω, with one common operator on all incidences of a gate; here $K = \mathbb{F}_4$, or its fixed extension $\mathbb{F}_{16}$. On each wire edge from u to v, implement $D_v D_u^{-1}$. If the supplied physical array at an input is f, define its logical value to be $D_{\text{source}}^{-1} f$. This defines an invariant and is not a physical preprocessing operation. Along an edge the stored value changes from $D_u g$ to $D_v g$. At a pointwise gate G, linearity and the common frame give

$$G(D_v g_1, \ldots, D_v g_q) = D_v G(g_1, \ldots, g_q).$$

Thus, if the scalar schedule routes role w to $\pi(w)$, the full physical operator on that route is

$$D_{\text{sink}(\pi(\text{w}))} D_{\text{source(w)}}^{-1}.$$

The identity holds on every data and dirty-scratch role. It also holds for a direct sum over active-mask sectors, since the frame within each sector is linear and each gate combines the same addresses.

Suppose a preceding Cartesian row coordinate has range divisible by W. Split rows cyclically as $q = Wg + (w-1)$ onto W role streams; each has the identical full Ω suffix and exactly $1/W$ of the volume. Execute the framed scalar network on them. At completion, merge output role $\pi(w)$ back into original row q, thereby undoing the scalar exchange. The array-frame identity proves the desired endpoint operator independently on every original row. Splits, merges and pointwise gates cost linear time in the current total volume, because their number and scalar degrees are fixed. This construction applies to arbitrary row ranges, not only powers of two.

For completeness, it uses a fixed set of tapes. Reserve the W role tapes, fixed I/O and work tapes, a parking stack, and a descriptor stack. Before visiting a child, append the inactive role streams to the parking stack in a fixed order, put the active stream into child I/O, and push only the parent descriptor and finite program counter. The child reuses the role/work tapes. On return, pop the inactive streams in reverse order, writing each destination from its known right end so its original direction is restored. Erase only the popped interval. Every push, pop, copy, reset and cleanup scan costs $O(V_0)$ at a parent of volume V_0; no descendant head crosses a parked ancestor. There are the fixed number s of child calls, each of volume V_0/W, so this is an $O(V_0)$ node toll, not an extra factor on the child cost. A row supply divisible by W^k permits exactly k such splits without further padding.

Address-control work is polynomial in the $O(h)$-bit key. Length and volume descriptors may additionally have $O(\log(R+1))$ bits. Their per-node arithmetic costs at most a fixed polynomial in $h + \log(R+1)$, and initializing a record counter costs $O(\log(R+1))$ per record. In the four-block use $R \geq D2^h$ above a fixed threshold, so $h \leq \log_2 R$ and every such fixed polynomial is $O(R)$. Record copying itself is $O(R)$ by amortized ripple counters, as proved in Lemma 7.2; no $\log R$ factor is charged per payload bit. These charges fit the current-node volume uniformly in D and at every role depth. Fixed small heights use direct finite circuits. This proves the role-array compiler used in Section 7 without importing a second machine oracle.

A.3 Factor geometry and gate labels

For the phase network take $h = 100$ and the factor space $F = \mathbb{F}_3^{h+1}$ with symmetric form

$$\langle (a,r),(b,s)\rangle = -\sum_{i=1}^{h} a_i b_i + rs.$$

For each triple S, let $t_S = (\mathbf{1}_S, 1)$ and $L_S = \langle t_S \rangle$. Then

$$\langle t_S, t_S\rangle = -3 + 1 = -2 = 1 \quad \text{in } \mathbb{F}_3, \qquad \langle t_S, t_T\rangle = 1 - |S \cap T|.$$

Thus each L_S is a nondegenerate unit line, and adjacent triples give orthogonal lines. The form is explicitly isometric to the standard dot product: on each pair of the first h coordinates use $(a,b) \mapsto (2a+2b, 2a+b)$ and leave the last coordinate unchanged. Indeed the sum of squares of the two new coordinates is $2a^2 + 2b^2$, the old paired form. The full-factor phase is therefore the same coordinatewise ternary phase operator on every factor axis.

Here is the label table for one invocation, with a common suffix factor suppressed. At a given stage let A be the prefix factor accumulated so far, let P be its distinguished prefix line, and write $A = B \perp P$ with $B = P^{\perp}$. For the current triple t, abbreviate $L = L_t$ and write $L^{\perp}$ for its orthogonal complement in F. Labels are subspaces; the symbol $\oplus^{\perp}$ denotes

an orthogonal direct sum.

wire role	label
t_0, Y-side	$B \otimes L$
t_1, Y-center	$B \otimes F$
t_2, X-side	$(B \otimes F) \oplus^{\perp} (P \otimes L)$
t_3, X-center	$A \otimes F$
t_4, Y-center	$B \otimes F$
t_5, Y-side	$(B \otimes F) \oplus^{\perp} (P \otimes L^{\perp})$
t_6, X-center	$A \otimes F$
t_7, X-side	$A \otimes F$

For an ordered adjacent side role (S, T), the actual successive labels are

$$\begin{aligned} B \otimes L_S &\subseteq (B \otimes F) \oplus^{\perp} (P \otimes L_T) \\ &\subseteq (B \otimes F) \oplus^{\perp} (P \otimes L_S^{\perp}) \subseteq A \otimes F. \end{aligned}$$

The middle inclusion is exactly $L_T \subseteq L_S^{\perp}$ from adjacency. All inclusions are orthogonal direct-summand inclusions, so the residual form is nondegenerate, not just a dimension count. The center sequence is $B \otimes F$, $A \otimes F$, $B \otimes F$, $A \otimes F$, giving its one downward transition. The data sequences use the corresponding common gate labels in the table. Together these describe every side, center and data edge. For example, the next prefix complement is

$$B' = (P \otimes L_t)^{\perp} = (B \otimes F) \oplus^{\perp} (P \otimes L_t^{\perp}),$$

which is exactly the t_5 label. These decompositions give a direct-sum proof of each within-invocation and interstage inclusion. In stage two the chronological physical gate order is the inverse forward order with logical source and target exchanged; the physical sequence remains Y-side, Y-center, X-side, X-center, Y-center, Y-side, X-center, X-side. This is why the labels are assigned to physical X, Y roles throughout, rather than relabeling the banks midway.

The physical endpoint labels must be distinguished from the phase frames. Each exceptional original X input has geometric source label U, its triple tensor line of dimension one; every original Y source and every scratch source has label 0. The scalar network routes original X data to the Y bank and original Y data to the X bank. Thus X sinks have label $F^{\otimes 3}$, the routed exceptional Y sinks have label $U^{\perp}$, and scratch sinks have label $F^{\otimes 3}$. The corresponding endpoint frames and routes are:

route	source label	source frame	sink label	sink frame
X data $\to Y$	U	C_U^{-1}	$U^{\perp}$	$C_{U^{\perp}}$
Y data $\to X$	0	I	$F^{\otimes 3}$	$C_{F^{\otimes 3}}$
scratch $\to$ scratch	0	I	$F^{\otimes 3}$	$C_{F^{\otimes 3}}$

In particular scratch is not assumed clean: its arbitrary input value is processed, and its full sink contribution is counted. The C_U^{-1} entry is an operator frame and is not a negative subspace label.

The signed endpoint dimension sum is the total sink dimension minus the total source dimension:

$$\big(Nm + N(m-1) + (W - 2N)m\big) - N = (Wm - N) - N = Wm - 2N.$$

Here the three sink terms are, respectively, the X bank, the Y bank, and scratch; the only nonzero source-label dimensions are the N exceptional X sources. This explicit route count explains the $-2N$ term without conflating geometric labels and phase matrices.

The dimensional telescope is short enough to state explicitly. Put $d = \dim F = h+1 = 101$ and $m = d^3 = 1{,}030{,}301$. A label tensor with factor dimension a has dimension ad at the next factor level. In the table the only downward transitions are the center transitions $t_3 \to t_4$, from $A \otimes F$ to $B \otimes F$; each loses d dimensions because A/B has dimension one. There are h center roles per invocation and I invocations, so the total downward loss is

$$L = Ihd.$$

The signed edge-dimension sum is the sink-minus-source total just computed, $Wm - 2N$. An absolute variation sum changes a decrease of size r from signed contribution $-r$ to absolute contribution $+r$, adding $2r$; hence its value is $Wm - 2N + 2L$. The phase-frame correction on the first edge of each exceptional route contributes one additional rank-one child. The total directional-child count is therefore

$$s = Wm - 2N + 2L + N = Wm - N + 2L.$$

The N-term is an endpoint correction, not a label-dimension drop and not a free frame change.

For $h = 100$, the exact values are

$$\begin{aligned} v &= 161700, \qquad z = 13968, \\ N &= 4{,}227{,}952{,}113{,}000{,}000, \qquad I = 78{,}440{,}670{,}000, \\ W &= 177{,}176{,}569{,}091{,}445{,}000{,}000, \\ L &= 792{,}250{,}767{,}000{,}000. \end{aligned}$$

The saving is certified by exact ratios

$$\frac{W}{N} = 41906 + \frac{1}{539}, \qquad \frac{N - 2L}{N} = \frac{337}{539}, \qquad 1 - \frac{s}{Wm} = \frac{337}{23271753837835}.$$

Since $m < 2^{20}$, we have $\log m < 20$ (using $\log 2 < 1$). The displayed deficit exceeds $20 \cdot 2^{-50}$, so for $\epsilon = 2^{-50}$,

$$1 - m^{-\epsilon} < \epsilon \log m < 20\epsilon < 1 - \frac{s}{Wm}.$$

It follows that $s/W < m^{1-\epsilon}$. This is the strict finite exponent inequality required by the role recurrence.

A.4 Phase directions at a growing node

The label count must be converted into directional children without conflating different norms. Let $K = \mathbb{F}_4 = \mathbb{F}_2(\eta)$, where $\eta^2 + \eta + 1 = 0$, and take the fixed three-point kernel

$$C = \begin{pmatrix} 1 & \eta^2 & \eta^2 \\ \eta^2 & 1 & \eta^2 \\ \eta^2 & \eta^2 & 1 \end{pmatrix}, \qquad C^3 = I.$$

For a nondegenerate ternary direction v, its norm $\nu = v \cdot v$ is either 1 or 2. A directional edge also has a sign $\epsilon \in \{1, -1\}$, according to its orientation in the signed frame difference. Its phase parameter is

$$\lambda = \epsilon/\nu \in \mathbb{F}_3^\times.$$

The corresponding directional operator is

$$C_{\langle v,\epsilon\rangle} = I + \eta^{-\lambda}(T_v + T_{-v}),$$

where η is the chosen primitive cube root in K and T_v translates by v. Fourier diagonalization gives the signed phase $\eta^{\epsilon(v\cdot x)^2/\nu} = \eta^{\lambda(v\cdot x)^2}$. Thus the child type is determined by λ, not by ν alone: reversing an edge changes λ to $-\lambda$ and inverts the phase. The two possible nonzero types are implemented by C and C^{-1}. Since $C^3 = I$ and coefficientwise $\mathbb{F}_4$ Frobenius conjugates C to C^{-1}, this costs one fixed wrapper around one child; it is not an additional recursive child.

At a growing node, write the selected address width as $e = mf$, where $m = d^3$ is the fixed slot width and f counts repeated coordinates. A residual direction from the fixed slot factor is a vector $v_j \in \mathbb{F}_3^m$. Its repeated direction is $v_j \otimes I_f$: the same slot coefficients act on each of the f coordinate axes. The child therefore has width f and applies the kernel specified by $\lambda_j = \epsilon_j/\nu_j$ uniformly to all those axes. A negative edge direction is not identified with a positive one merely because its norm is unchanged. This is one width-f child for the fixed direction v_j and signed edge, not f separate children.

An orthogonal basis of the residual space may contain both signs and both norms. Each signed basis direction is a separate child, already counted in the rank sum s with its edge orientation. Its ϵ_j and norm ν_j are fixed across all repeated axes of that child. One must not posit a single global Frobenius wrapper that inverts an arbitrary subset of axes: such a mixed subset need not be one directional child. The valid argument is coordinatewise by the orthogonal basis: for each j, use its one width-f child with type $\lambda_j = \epsilon_j/\nu_j$. Thus the signed types affect which fixed wrapper each counted child uses, but do not alter the count or introduce hidden children.

Finally, for each exceptional data route let U be its triple tensor line. The source-frame choice is C_U^{-1}, with the other source frames the identity and the prescribed positive phase frames at gates and sinks. Because $U \perp U^\perp$ and C_U has order three, the routed endpoint identity is

$$C_{U^\perp}(C_U^{-1})^{-1} = C_{U^\perp}C_U = C_{F^{\otimes 3}}.$$

At the first X edge, the chosen head frame C_U and source frame C_U^{-1} give edge operator $C_U(C_U^{-1})^{-1} = C_U^2 = C_U^{-1}$, a rank-one directional correction. There are exactly N such exceptional data sources, which is the added N in $s = Wm - N + 2L$. The ordinary Y-source to X-sink routes and all scratch routes start with frame $C_0 = I$ and end with $C_{F^{\otimes 3}}$. Thus every routed path has the required full phase after the correction, while the geometric sink label on the exceptional Y endpoint remains $U^\perp$. Scratch values remain arbitrary throughout; the endpoint statement is an operator identity on the entire role space.

The scalar schedule, direct-sum labels, dimensional telescope, and source correction together provide the finite certificate used at every growing node. The asymptotic tape cost still depends on the separately stated fixed-tape interchange/router implementation and the recurrence that consumes the strict inequality; this appendix does not replace those machine-level arguments.

B An explicit positive word for the binary tensor core

This appendix gives a fixed matrix identity over a small extension field. It will be used only as an algebraic identity: by itself it implies no asymptotic running-time bound. Let

$$\mathbb{F}_{16} = \mathbb{F}_2[t]/(t^4 + t + 1), \qquad \eta = t^5 = t^2 + t.$$

The polynomial is irreducible: it has no root in $\mathbb{F}_2$, and reduction modulo the only irreducible quadratic $X^2 + X + 1$ gives remainder 1. Consequently the quotient is a field. The relation $t^4 = t + 1$ gives

$$\eta^2 + \eta + 1 = t^4 + t + 1 = 0, \qquad \eta^3 = 1,$$

so $\mathbb{F}_4 = \mathbb{F}_2(\eta)$ is its subfield. Put $a = \eta^2$ and $b = \eta$; then $ab = 1$. The element t is outside $\mathbb{F}_4$, and in particular t, $t+1$, and t^2+t+1 are nonzero.

All matrices below act on column vectors, and products are composed rightmost first. Write J for the 3-by-3 all-ones matrix and define

$$C_3 = \eta I_3 + \eta^2 J = \begin{pmatrix} 1 & a & a \\ a & 1 & a \\ a & a & 1 \end{pmatrix}, \qquad A = C_3 \oplus [1].$$

Since $J^2 = J$ in characteristic two, direct multiplication gives $C_3^2 = \eta^2 I_3 + \eta J$ and $C_3^3 = I_3$. Thus $A^{-1} = A^2$. The only nonmonomial letter permitted below is the forward matrix A; every inverse occurrence is replaced by two forward occurrences. Diagonal matrices with nonzero entries and coordinate permutations are monomial.

Projectors and a rank-one commutator. Let E_{ij} denote the standard matrix units and set

$$u = (1, a, a, 0)^T, \quad v = (1, b, b, 0)^T, \quad P = AE_{11}A^{-1} = uv^T.$$

Indeed, the first column of A is u, and the first row of A^{-1} is v^T. Also $v^T u = 1 + ab + ab = 1$, hence $P^2 = P$. With

$$D = \operatorname{diag}(t, 1, t+1, 1), \quad Q = DPD^{-1} = x_0 y_0^T,$$

where

$$x_0 = (t, a, a(t+1), 0)^T, \qquad y_0 = (t^{-1}, b, b/(t+1), 0)^T,$$

we again have $Q^2 = Q$. Calculation in $\mathbb{F}_{16}$ gives

$$v^T x_0 = t + 1 + (t+1) = 0, \qquad y_0^T u = t^{-1} + 1 + (t+1)^{-1} = \kappa, \quad \kappa = \frac{t^2+t+1}{t(t+1)} \neq 0.$$

Therefore $PQ = 0$ while $QP = \kappa x_0 v^T$.

For idempotents P, Q satisfying $PQ = 0$, expansion using $(I + cP)^{-1} = I + \frac{c}{1+c}P$ yields

$$[I + cP, I + dQ] := (I + cP)(I + dQ)(I + cP)^{-1}(I + dQ)^{-1} = I + \frac{cd}{1+c}QP,$$

when the displayed inverses exist. To see the cancellation, multiply the first three factors and use $PQ = 0$: the P terms cancel, leaving $I + dQ + \frac{cd}{1+c}QP$. On multiplication by $(I + dQ)^{-1}$, the Q term cancels and the extra term vanishes because $QPQ = Q(PQ) = 0$.

Define the diagonal $R = \operatorname{diag}(\eta, 1, 1, 1)$ and let

$$G = ARA^{-1} = I + aP, \qquad H = DGD^{-1} = I + aQ.$$

The formula above with $c = d = a$ has coefficient $a^2/(1+a) = \eta^4/(1+\eta^2) = 1$. Thus

$$T_0 = GHG^{-1}H^{-1} = I + QP = I + \kappa x_0 v^T.$$

This is already a positive word: each of G, H, G^{-1}, H^{-1} uses three forward A letters after replacing A^{-1} by A^2. Hence T_0 uses 12 forward A letters; its other letters are monomial.

Four successive transvections. Let $B = \mathrm{diag}(1, \eta, \eta, 1)$. Then

$$x = Bx_0 = (t, 1, t+1, 0)^T, \qquad w^T = v^T B^{-1} = (1, 1, 1, 0), \quad T_1 = BT_0B^{-1} = I + \kappa x w^T.$$

Let S_{12} interchange coordinates 1 and 2. It fixes the row w^T and $w^T S_{12} x = 1 + t + (t+1) = 0$. Consequently, multiplying the two rank-one updates in $T_1 S_{12} T_1 S_{12}^{-1}$ gives

$$T_2 = I + \kappa(t+1)(e_1 + e_2)w^T.$$

In this calculation the two cross terms combine as $\kappa(x + S_{12}x)w^T$; the quadratic term is zero because $w^T x = 0$. Now take $D_3 = \mathrm{diag}(1, 1, \eta, 1)$. The row difference $w^T + w^T D_3^{-1}$ is $(1 + \eta^{-1})e_3^T = \eta e_3^T$, and both row vectors pair to zero with $e_1 + e_2$. Hence

$$T_3 = T_2 D_3 T_2 D_3^{-1} = I + \kappa(t+1)\eta(e_1 + e_2)e_3^T.$$

With $D_1 = \mathrm{diag}(\eta, 1, 1, 1)$, the corresponding rank-one update has zero cross-pairing; its column difference is $(1 + \eta)e_1 = \eta^2 e_1$. Therefore

$$T_4 = T_3 D_1 T_3 D_1^{-1} = I + \kappa(t+1)E_{13}.$$

Put $\mu = \kappa(t+1) \neq 0$. Diagonal conjugation removes this coefficient:

$$T_5 = \mathrm{diag}(\mu^{-1}, 1, 1, 1)\, T_4\, \mathrm{diag}(\mu, 1, 1, 1) = I + E_{13} =: U_{13}(1).$$

Here $U_{ij}(c) = I + cE_{ij}$ is a transvection. At each of the four commutator stages the word length doubles, while the conjugating diagonal adds two monomial letters. The forward-A counts are therefore $12, 12, 24, 48, 96, 96$ for $T_0, T_1, T_2, T_3, T_4, T_5$, respectively. The final word for $U_{13}(1)$ has 192 letters, 96 of them equal to A. Conjugating by a permutation gives a positive word for any desired $U_{ij}(1)$, with the same 96 forward-A occurrences and two additional permutation letters except for the base pair.

The target and its factor order. Write

$$H_2 = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = U_{12}(1)S,$$

where S swaps the two coordinates. Let $P_f = (1\ 3)(2\ 4)$ interchange the first-coordinate labels and $P_s = (1\ 2)(3\ 4)$ interchange the second-coordinate labels. With coordinates ordered $(1,1), (1,2), (2,1), (2,2)$, the exact rightmost-first identity is

$$H_2 \otimes H_2 = U_{13}(1)U_{24}(1)P_f\, U_{12}(1)U_{34}(1)P_s.$$

The first two transvections are the two disjoint copies of U_{12} on the first tensor factor, and the next two are the copies on the second; the intervening and final permutations implement the two swaps. Equivalently the combined wire permutation is $P_f P_s = (1\ 4)(2\ 3)$, and the four transvections are all over $\mathbb{F}_2$. Replace each $U_{ij}(1)$ by the explicit conjugate of T_5 just constructed. The resulting word has exactly $4 \cdot 96 = 384$ forward A letters. The four U words contribute 768 letters before conjugating permutations; the three non-base conjugations add six permutation letters, and P_f, P_s add two more. Thus the full positive word has 776 letters, of which 392 are monomial. Its product is exactly $H_2 \otimes H_2$, whose entries lie in $\mathbb{F}_2$.

A separate bounded witness checks the chosen coordinates and finite identities; it is corroboration for this calculation, not a proof of a runtime bound. That witness is not included in this focused bundle. The word alone does not establish a tensor-rank saving, a fast A-tensor algorithm, or an asymptotic multiplication theorem. Those claims require separate arguments for tensoring the word, routing its monomial factors, and charging every operation in the chosen machine model.

References

[1] Alin Bostan and Éric Schost. Polynomial evaluation and interpolation on special sets of points. *Journal of Complexity*, 21(4):420–446, 2005. doi:10.1016/j.jco.2004.09.009. Author-hosted paper: https://cs.uwaterloo.ca/~eschost/publications/BostanSchost.pdf.

[2] Josh Alman. Faster Walsh-Hadamard transform and matrix multiplication over finite fields using lookup tables. In *Symposium on Simplicity in Algorithms (SOSA)*, pages 137–144, 2023. doi:10.1137/1.9781611977585.ch13. Author preprint: https://arxiv.org/abs/2211.04643.

[3] David Harvey and Joris van der Hoeven. Faster polynomial multiplication over finite fields using cyclotomic coefficient rings. *Journal of Complexity*, 54:Article 101404, 2019. doi:10.1016/j.jco.2019.03.004. Author-hosted paper: https://www.texmacs.org/joris/cyclomult/cyclomult.pdf.

[4] David Harvey and Joris van der Hoeven. Polynomial multiplication over finite fields in time $O(n \log n)$. *Journal of the ACM*, 69(2), Article 12, pages 1–40, 2022. doi:10.1145/3505584. Author-hosted paper: https://www.texmacs.org/joris/ffnlogn/ffnlogn.pdf.

[5] David Harvey, Joris van der Hoeven, and Grégoire Lecerf. Faster polynomial multiplication over finite fields. *Journal of the ACM*, 63(6), Article 52, 2017. doi:10.1145/3005344. Author-hosted paper: https://www.texmacs.org/joris/ffmul/ffmul.pdf.

[6] David Harvey and Joris van der Hoeven. Integer multiplication in time $O(n \log n)$. *Annals of Mathematics*, 193(2):563–617, 2021. doi:10.4007/annals.2021.193.2.4. Publisher page: https://annals.math.princeton.edu/2021/193-2/p04.

[7] OpenAI. *Integer multiplication below $n \log n$.* Research preprint, 23 September 2026, Sections 3–4; Lemma 12 supplies the arbitrary-width equal-chunk interchange statement. Citation revision `4eaee87548250b2682affda7d05d947c05554bae`, inspected 31 October 2026. https://github.com/openai/math/blob/4eaee87548250b2682affda7d05d947c05554bae/preprints/Integer-multiplication-below-n-log-n-September-23-2026/build/sections/04-swap.tex.

[8] David G. Cantor. On arithmetical algorithms over finite fields. *Journal of Combinatorial Theory, Series A*, 50(2):285–300, 1989. https://doi.org/10.1016/0097-3165(89)90020-4.

[9] Shuhong Gao and Todd Mateer. Additive Fast Fourier Transforms over Finite Fields. *IEEE Transactions on Information Theory*, 56:6265–6272, 2010. https://doi.org/10.1109/TIT.2010.2079016. Author manuscript: https://www.math.clemson.edu/~sgao/papers/GM10.pdf.

[10] Chunlei Liu. *Multiplication of polynomials over the binary field.* arXiv:2505.03101v2, 14 May 2025. https://arxiv.org/abs/2505.03101v2.

[11] A. Karatsuba and Yu. Ofman. Multiplication of many-digital numbers by automatic computers. *Doklady Akademii Nauk SSSR*, 145(2):293–294, 1962 (Russian original). https://www.mathnet.ru/php/archive.phtml?jrnid=dan&option_lang=eng&paperid=26729&wshow=paper.

[12] David Harvey. *On the recently proposed $o(n \log n)$ integer multiplication algorithm.* Unpublished discussion note, 20 October 2026, Section 4. https://web.maths.unsw.edu.au/~davidharvey/research/subnlogn.pdf.

[13] Nicolas Bonamy and Derrick Choi. Codex as a platform: build on the open agent harness. OpenAI Developers, 19 August 2026. Official documentation reference for the Codex software used in release preparation and source inspection; accessed 31 October 2026. https://developers.openai.com/blog/codex-as-a-platform.